Computer Networks 285 (2026) 112375

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

RankShield: A hybrid deep learning and threshold-based framework for the detection and mitigation of RPL rank attacks in IoT networks

Eden Teshome Hunde [a,b], Shereen Ismail [c,*]

[a] Department of Electronics and Informatics (ETRO), Vrije Universiteit Brussel, Pleinlaan 2, 1050 Brussels, Belgium
[b] Department of Electrical and Computer Engineering (ECE), Jimma Institute of Technology, Jimma University, 378 Jimma, Ethiopia
[c] Merit Network, Inc., University of Michigan, Ann Arbor, MI 48108, USA

ARTICLE INFO

Keywords:
RPL
DNN
Rank attack
Intrusion detection and mitigation
IoT
Energy-efficient security

ABSTRACT

The Internet of Things (IoT) has become a fundamental component of critical sectors such as healthcare, smart cities, and industrial operations. In these environments, even a minor security compromise could lead to serious consequences such as data latency, interruption of services, or even physical harm. Among IoT routing protocols, the IPv6 Routing Protocol for Low-Power and Lossy Networks (RPL) is particularly vulnerable to rank attacks (including increased and decreased rank attacks). The fundamental motivation for focusing on rank manipulation is that it exploits the core logic of the RPL topology; unlike resource-flooding attacks that cause temporary congestion, rank attacks permanently corrupt the network's logical hierarchy. This vulnerability stems from RPL's fundamental reliance on a self-reported "rank" metric to maintain its topological structure without a centralized verification authority. While RPL is susceptible to various threats, including Sybil, blackhole, and version number attacks, rank manipulation is uniquely damaging because it directly corrupts the network's logical hierarchy, leading to widespread routing instability. This research is motivated by the need to secure IoT environments against such threats, especially given the resource constraints inherent in many IoT devices. Accordingly, this paper introduces RankShield, a novel hybrid, energy-efficient and decentralized framework for detecting and mitigating RPL rank attacks in resource-constrained IoT environments.

RankShield integrates two lines of defence: deep neural network (DNN)-based rank detection mechanism, that identifies and classifies abnormal rank behavior, and a threshold-based inconsistency-checking mechanism for validation and mitigation, which confirms the predictions of the DNN model and validates malicious activity, thereby reducing unnecessary alarms. Detected attackers are further isolated through blacklist propagation, preventing compromised nodes from rejoining the network. To evaluate RankShield, we used the Cooja simulator to generate a comprehensive dataset, enabling the DNN model to learn both increased and decreased rank attack patterns under realistic conditions. The experimental results show that RankShield reduces latency by 66% and energy consumption by 45%, while improving the packet delivery ratio (PDR) by 67%. Furthermore, RankShield outperforms baseline RPL and existing intrusion detection approaches, including SBIDS and ARM. By combining data-driven detection with lightweight rule-based validation, RankShield improves the security and reliability of IoT networks and provides a basis for future research on hybrid security mechanisms.

1. Introduction

The proliferation of the Internet of Things (IoT), powered by advances in wireless technology, has reshaped key sectors such as smart cities, healthcare, and industrial automation [1]. The IoT envisions a future where smart, interconnected devices form an ecosystem that enriches daily life through autonomous communication and intelligent, context-aware services. Low-Power and Lossy Networks (LLNs) are a vital part of many IoT systems. However, these networks consist of devices with limited power, memory, and processing abilities, which present unique security challenges. The vast volume of sensitive data exchanged across these resource-constrained devices in LLNs, as part of the broader IoT network, makes them particularly vulnerable to routing-based attacks, where adversaries exploit weaknesses in the underlying communication protocols. Furthermore, the increasing adoption of encrypted protocols like DNS over HTTPS (DoH) creates new

* Corresponding author.
E-mail addresses: eden.teshome.hunde@vub.be, eden.teshome@ju.edu.et (E.T. Hunde), drismail@umich.edu (S. Ismail).

https://doi.org/10.1016/j.comnet.2026.112375
Received 6 January 2026; Received in revised form 16 April 2026; Accepted 5 May 2026
Available online 7 May 2026
1389-1286/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

blind spots for traditional security measures, necessitating advanced, privacy-preserving defense frameworks [2,3]. Recent research has investigated federated versus decentralized architectures to provide energy-efficient and scalable intrusion detection for both industrial IoT and encrypted traffic environments [4,5]. These emerging threats underscore the need for hybrid, decentralized architectures capable of identifying sophisticated network anomalies while maintaining the resource efficiency required by constrained devices. Among these protocols, the IPv6 Routing Protocol for Low-Power and Lossy Networks (RPL) is the de facto standard for IoT routing. While RPL is lightweight and well-suited for constrained environments, its reliance on rank values, a metric used to determine a node's position within the routing topology, creates a critical vulnerability. RPL is uniquely susceptible to rank-based exploitation because it operates as a distributed, trust-based protocol where nodes independently calculate and broadcast their distance from the root. The protocol lacks a native, lightweight mechanism to cross-verify these claims, allowing a single compromised node to influence the parent-selection process of its entire neighborhood. Moreover, nodes assume that neighbors are reporting their distance to the root (rank) honestly to minimize control overhead. However, in adversarial environments, this lack of validation allows a single compromised node to trigger a "topological domino effect." While other RPL-specific threats such as DIS flooding or Destination Advertisement Object (DAO) inconsistency focus on resource exhaustion or local table corruption, rank attacks target the core topological stability of the Destination-Oriented Directed Acyclic Graph (DODAG). By manipulating this single value, an attacker can induce routing loops or position themselves as a "sinkhole" or "wormhole," making it a high-impact "gateway" attack that facilitates further malicious activity.

Rank attackers can manipulate rank values within the RPL routing protocol to mislead neighboring nodes, disrupt routing paths, influence network topology and traffic flow, and degrade overall network performance. These so-called rank attacks can take the form of decreased rank attacks, where a malicious node advertises a lower rank than it legitimately has to become the root node or preferred parent, potentially attracting traffic. Conversely, an increased rank attack occurs where the attackers broadcast an artificially higher rank to disrupt parent-child relationships and cause other nodes to choose it as a parent. While many RPL attacks, such as selective forwarding or sinkhole attacks, may exhibit similar external signs (i.e., dropped packets or high latency), rank attacks are the root cause of the structural deviations that enable these behaviors. Both variants of rank attacks can cause routing loops, excessive energy consumption, reduced packet delivery, and even complete denial of service.

RPL is vulnerable to such types of attacks primarily due to its stateless and distributed nature, coupled with the critical role of rank in its routing mechanism. Unlike centralized networks where a controller validates topology, nodes in RPL independently determine their parents based on observed rank values without a centralized authority or strong mechanisms to verify the authenticity and integrity of these ranks. Malicious nodes can easily inject false rank information (i.e., in the form of increased or decreased rank), which propagates through the network. This leads to widespread routing disruption. Furthermore, the objective function, which dictates how ranks are calculated, often relies on simple hop counts or link quality indicators that are easily manipulable.

The existing RPL standard (RFC 6550) [6] requires significant revision as it lacks built-in mechanisms to verify the authenticity or consistency of advertised rank values. This deficiency has led to the proposal of numerous countermeasures, including anomaly detection methods that monitor parent change frequency or control packet behavior. Other approaches involve trust-based systems, where nodes assign trust scores to neighbors based on past interactions, and reputation systems that penalize misbehaving nodes. Cryptographic techniques, such as hash chains or digital signatures, have also been suggested to secure routing messages. Furthermore, centralized Intrusion Detection Systems (IDS) have been proposed, where a single node validates incoming control messages. Threshold-based mechanisms involve setting predefined thresholds for certain metrics of the ranked data, while a Machine Learning (ML)-based approach typically involves training a model to identify anomalous ranking patterns that are indicative of an attack.

Despite the breadth of these studies, several critical research gaps remain. First, most current solutions operate as "single-line" defenses, relying exclusively on either rigid, rule-based thresholds, which are prone to high false-positive rates during network congestion or complex Deep Learning models that can be computationally expensive for individual IoT motes. Second, a significant gap exists in the comprehensive detection of both decreased and increased rank attacks; most existing DNN-based methods [7–9] target only decreased rank scenarios, leaving the network vulnerable to topological disruptions caused by artificially inflated (increased) ranks. Finally, centralized IDS architectures create a single point of failure and introduce significant communication latency, which is unsuitable for large-scale, dynamic IoT deployments.

The broader research challenge in securing RPL networks lies in the trade-off between rigorous security and the extreme resource constraints of IoT motes. Implementing heavy cryptographic verification at every hop would exhaust the battery life of nodes, yet relying on simple thresholds is insufficient to catch sophisticated, adaptive rank attackers.

To address this challenge, we present RankShield, a hybrid, energy-efficient, decentralized detection and mitigation framework designed to counter RPL rank attacks in resource-constrained IoT networks. RankShield distinguishes itself from existing literature by integrating two distinct lines of defense: a lightweight DNN-based detection model for rapid classification and a subsequent threshold-based inconsistency-checking mechanism for rigorous validation. This proposed approach integrates a DNN-based detection model to identify abnormal rank behavior with a threshold-based inconsistency-checking mechanism to validate detections and confirm malicious activity. Once identified, attackers are mitigated through threshold-based strategies and isolated via blacklist propagation, preventing re-entry into the network. Accordingly, this research offers the following core contributions:

- A decentralized, two-step detection architecture that mitigates the risk of single-point failure while reducing false alarms through multi-stage validation.
- Two-step rank attack detection that combines DNN-based classification with a threshold-based inconsistency-checking mechanism for detection validation.
- A comprehensive mitigation strategy that addresses both increased and decreased rank attacks, a dual capability often missing in current protocols.
- Dynamic mitigation using a monitoring table that flags and tracks suspicious nodes.
- Attack isolation using a blacklist mechanism that prevents reintegration of compromised nodes and alerts neighboring RPL nodes to avoid them.

Unlike conventional, single-line intrusion detection approaches [10–12], RankShield introduces a hybrid (two-step) detection approach to identify RPL rank attacks. The first line of defense employs a DNN model to classify incoming rank attacks as either increased or decreased. The second step applies a dynamically adjusted, threshold-based inconsistency-checking mechanism to confirm the DNN's predictions and validate malicious activity based on incoming DIO messages. By combining data-driven insights with rule-based verification, RankShield addresses the gap between detection accuracy and resource efficiency. The proposed framework operates in a decentralized manner, enabling individual nodes to analyze DIO messages locally for the detection of both increased and decreased rank attacks. This approach improves detection accuracy, reduces false positives and energy consumption, and supports reliable and secure routing in resource-constrained IoT deployments. The evaluation of RankShield is conducted exclusively

through simulations, and hardware-based verification is outside the scope of this work.

The remainder of this paper is organized as follows. Section 2 reviews related work and discusses existing security challenges, with a focus on rank attack detection and mitigation. Section 3 describes the fundamental operations of RPL and provides an analysis of rank attacks. Section 4 presents the proposed RankShield framework. Section 5 details the simulation setup, results, and performance evaluation. Finally, Section 6 concludes the paper and outlines directions for future work.

2. Literature review

The widespread adoption of modern IoT applications has introduced numerous potential security risks, making routing attacks a critical ongoing area of investigation, particularly for RPL-based IoT networks [13]. The primary challenge identified in current literature is that RPL attacks often exhibit overlapping signs; for instance, both rank attacks and Sybil attacks can result in high packet loss. However, the root cause of rank attacks is specifically the exploitation of the Objective Function (OF), which is the mathematical engine of the protocol. Most existing countermeasures focus on "symptom-based" detection (monitoring packet drops), whereas securing the rank calculation process itself addresses the source of the structural instability.

RPL is vulnerable to a broad spectrum of routing and resource attacks, including Forwarding Misbehavior and Selective Forwarding, which compromise data integrity [14]; DAO Inconsistency and DIO/DIS Suppression, which disrupt topology maintenance [15,16]; and Sybil or DIS-Spam attacks that exhaust network resources through identity fabrication or control-message flooding [17,18]. More advanced attacks, such as Hatchetman (worst-parent/child behaviors) and Vampire attacks, specifically target energy depletion and routing instability [19]. While existing countermeasures summarized in Table 1, ranging from cryptographic HMAC protections and trust-based systems to next-generation federated learning and neuromorphic AI frameworks [3,5], have improved the reliability of RPL control traffic, they often incur prohibitive overhead or fail to provide the multi-stage validation required for decentralized environments. Among these threats, rank attacks (e.g., increased and decreased rank manipulation) represent a particularly critical challenge. Unlike resource-focused attacks (like DIS-spam) that target node availability, rank attacks compromise the integrity of the routing path itself. Securing the network against rank manipulation is significantly more complex than other RPL-specific attacks, as it requires the detection mechanism to distinguish between legitimate rank adjustments, driven by physical mobility or environmental interference, and deliberate malicious activity Fig. 1.

Recent years have seen a surge of ML and hybrid IDS approaches tailored for IoT/RPL environments. Hybrid DL/ML frameworks that combine, for example, recurrent neural networks (RNNs) and gated recurrent units (GRUs), convolutional neural networks (CNNs) or ensemble classifiers with anomaly thresholds have been proposed and demonstrate improved detection accuracy with heterogeneous IoT datasets [20–22]. Other studies apply Random Forest (RF)-based reinforcement learning enhancements specifically to RPL to improve QoS and resilience while integrating security checks [23]. Federated learning has also been explored to detect selective forwarding and related forwarding attacks without centralizing raw data [24,25].

Recent surveys and systematic reviews highlight the shift toward hybrid AI + rule-based approaches to balance accuracy and resource-efficiency in constrained IoT systems and call for greater evaluation on multiple attack classes and real-world testbeds [26–28]. These trends motivate our choice of a hybrid DNN + threshold design, in which the DNN captures complex/non-linear attack patterns while lightweight thresholds and inconsistency checks reduce false positives and runtime overhead.

We clarify that RankShield focuses solely on rank inconsistency attacks (both increased and decreased rank). Extending this hybrid architecture to address forwarding misbehaviour, DAO inconsistency, Sybil, or energy depletion attacks will require additional data collection, feature engineering (e.g., per-flow forwarding statistics, DAO verification fields), and possibly separate detection modules. Such extensions are outside the current manuscript's scope but are a logical direction for future work. The remainder of this section reviews existing research on detecting and mitigating rank attacks in RPL-based IoT networks.

In [7], the authors propose a secure parent selection scheme in which child nodes evaluate neighboring parents based on a rank threshold defined between the maximum and average advertised ranks. Neighbors advertising significantly lower ranks are considered potential parents, reducing the likelihood of associating with malicious nodes. Experimental results demonstrate improved resistance to rank-based attacks. However, the effectiveness of the scheme is highly sensitive to threshold selection. An overly strict threshold may exclude legitimate parents with slightly higher yet acceptable ranks, leading to increased path stretch or latency, whereas a loose threshold may still permit sophisticated attackers that manipulate ranks within acceptable bounds. Additionally, frequent rank fluctuations by malicious nodes can increase control message overhead, as rank discovery and advertisement remain unchanged.

In [8], the authors introduce the Authentication Rank and Routing Metric (ARM), a hybrid specification-based IDS for RPL. ARM employs a centralized–distributed architecture in which the sink node analyzes

Table 1
Review for selected RPL attacks, representative countermeasures, and their limitations.

RPL Attack	Threat Definition	Representative Countermeasures	Main Limitations of Countermeasure
Decreased / Increased Rank (rank inconsistency)	Manipulation of RPL rank to disrupt routing, create loops, or cause network instability	Threshold checks (e.g., RTT/Δrank), trust models, specification-based monitors, ML classifiers	Thresholds alone can be evaded or cause false positives; trust systems may be slow or require extra messaging; centralized schemes create SPOF
Forwarding misbehaviour / Selective forwarding	Nodes selectively drop or misroute packets, reducing delivery ratio	Watchdog/monitoring nodes, multipath routing, trust/reputation systems, ML anomaly detection	Monitoring is energy-expensive; multipath increases overhead; detection can be delayed
DAO inconsistency	Injection of inconsistent DAO messages to corrupt routing tables	DAO validation schemes, consistency checks at sink, protocol patches	May require additional DAO traffic or centralized checking; limited scalability
DIO/DIS suppression & spam DIS	Malicious nodes suppress DIO/DIS messages or flood spam DIS to disrupt network formation	Rate-limiting, FSM/specification monitors, cryptographic authentication	Rate limiting may split legitimate re-joins; cryptography is costly for constrained nodes
Energy-depletion / Vampire	Attackers exhaust node energy by repeated control messages or forwarding	Rate limits, authentication, behaviour anomaly detection	Hard to distinguish malicious from heavy benign workloads; signatures are limited
Sybil attack	Nodes forge multiple identities to influence routing or bypass trust mechanisms	Identity validation, resource tests, trust, cryptographic IDs	Strong identity schemes require PKI or bootstrapping; resource tests can be spoofed
Hatchetman / Worst-parent	Attackers force victims to select poor parents repeatedly, degrading network performance	Thresholds on parent selection, parent diversity, trust	Can mitigate symptoms but can be bypassed by coordinated attacks

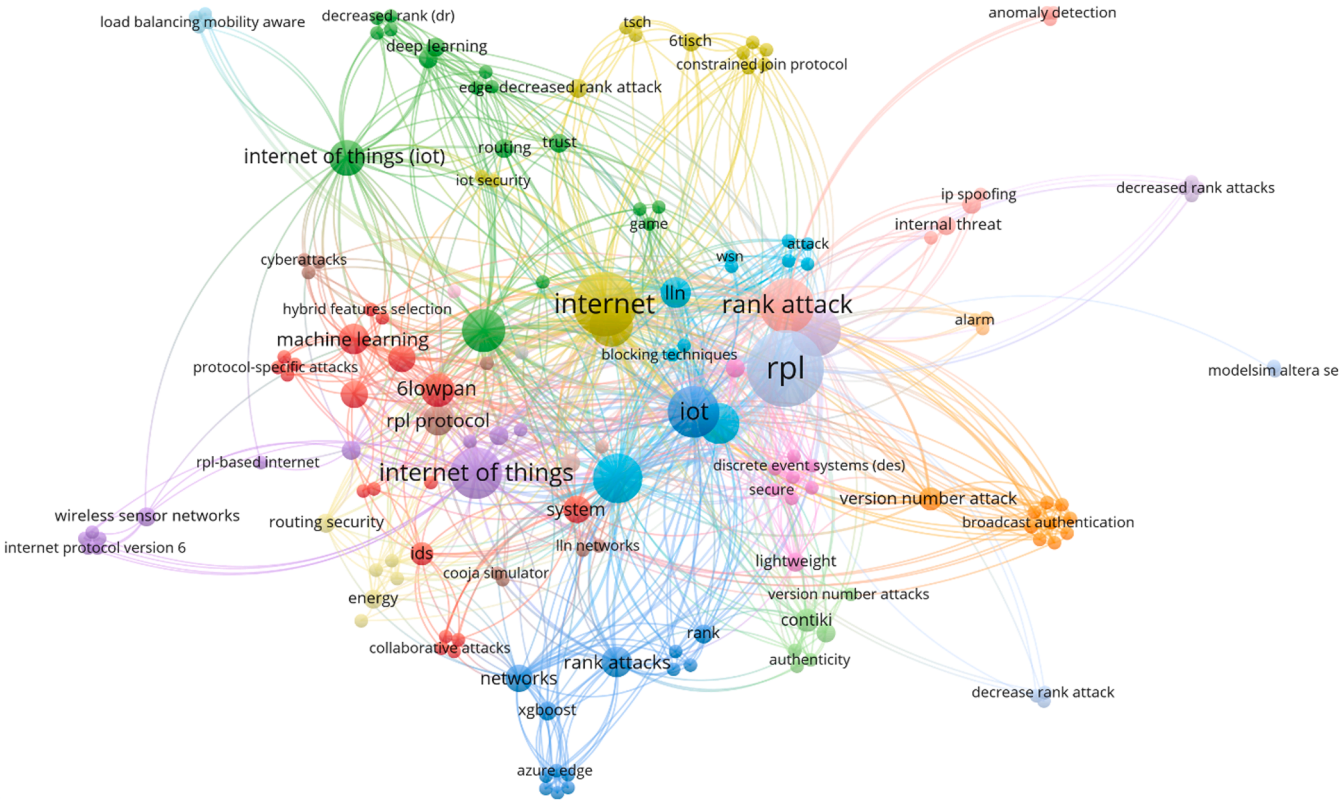


Fig. 1. Keyword co-occurrence map of RPL and IoT Security Research.

DIO messages and performs authentication and routing metric validation, while distributed nodes report detected anomalies. The scheme achieves high detection accuracy; however, its strong reliance on the sink node represents a critical limitation. Failure or compromise of the sink—due to attack, hardware fault, or energy depletion—can render the network vulnerable to multiple routing attacks. Furthermore, ARM suffers from scalability issues, as increased network size leads to higher processing and communication overhead at the sink, potentially causing detection delays, congestion, and degraded network performance.

In [29], the authors propose E2V, a centralized technique for detecting, mitigating, and isolating rank inconsistency attacks in RPL-based IoT networks. Implemented at the root node, E2V operates in three phases—rank calculation, substantiation, and attacker elimination—and leverages node energy consumption patterns to identify rank anomalies. Experimental results show reductions in energy consumption, attacker identification delay, and network convergence time. However, E2V's centralized design introduces a single point of failure: compromising or disabling the root node renders the detection mechanism ineffective. Moreover, aggregating rank and energy information from large-scale networks at the root can lead to significant processing overhead and detection latency. The reliance on energy-based indicators further limits robustness, as attackers may falsify energy reports, while legitimate workload fluctuations can trigger false positives.

In [30], the authors introduce the Rank Attack Detection (RAD) algorithm, a lightweight and energy-efficient approach that integrates non-cryptographic hashing to ensure control-packet integrity and random sampling to reduce energy consumption. RAD is evaluated across multiple topologies and demonstrates improved accuracy and lower energy usage, particularly in grid-centered deployments. However, the use of random sampling inherently weakens detection reliability, as short-lived or sporadic rank attacks may go undetected depending on the sampling rate and network size, highlighting a trade-off between energy efficiency and detection accuracy.

Secure-RPL (SRPL) is presented in [31], as a threshold-based mechanism that mitigates rank attacks by monitoring abnormal increases in advertised rank values and blocking malicious nodes from advancing within the DODAG. While SRPL improves routing stability under attack conditions, its practical deployment on resource-constrained devices is challenging, as the incurred monitoring overhead may outweigh the benefits when attack impact is limited.

To address the overhead present in [31], the authors in [32] developed and proposed a Time-Based Trust-Aware RPL (SecTrust-RPL) to offer secure protection against both rank and Sybil attacks. This protocol provides attack detection and isolation while optimizing network performance. In SecTrust-RPL, each node calculates its trustworthiness based on direct and recommended trust values from its neighbors. The evaluation results indicate that the proposed protocol significantly enhances protection against decreased rank attacks; however, it does not address the challenge of increased rank attacks.

In [33], the authors designed a Specification-Based IDS. This system employs Finite State Machine (FSM) transitions for attack detection, and its monitoring architecture involves the formation of Monitoring Nodes (MNs). To detect rank attacks, MNs scan for malicious nodes advertising lower ranks. The MNs identify discrepancies between the expected and fabricated ranks of these malicious nodes, initiating information cross-checking to determine valid ranks. However, the authors' approach to securing RPL networks against rank attacks primarily relies on a continuous monitoring mode. This constant vigilance leads to rapid energy depletion in the nodes, consequently reducing the network's lifespan. This makes the system less sustainable and challenging for long-term practical deployments where energy efficiency is paramount Table 2.

3. Background

3.1. RPL protocol

RPL is a proactive IPv6 distance-vector routing protocol

Table 2
A summary of recent work on rank attacks and their mitigation strategies.

Ref.	Mechanism	Detection Approach	Research Contribution	Limitations	Validation Tools	Supports One-Step Detection Mechanism	Support Mobility
[7]	Threshold-based	Secure Parent, Node	Enables child nodes to identify legitimate parents and avoid malicious nodes, using a fully distributed detection approach.	Vulnerable to blackhole and Sybil attacks and may lead to suboptimal routing paths. The mechanism is limited to detecting decreased rank attacks only.	Cooja, Simulator	✓	×
[8]	Specification-based IDS.	ARM	Consists of a centralized sink module and distributed modules on RPL nodes to prevent intruders from becoming successors or time sources, protecting synchronization in 6TiSCH networks. It supports a centralized and distributed approach.	Attack containment strategies are not addressed. The system is limited to detecting decreased rank attacks only.	Cooja Simulator	✓	×
[9]	ML based	KNN based, Approach	Proposes MLTKNN, a K-nearest neighbor-based method that detects rank attacks by comparing declared rank with actual rank relative to the border router; shown to perform well in terms of delay, PDR, and detection accuracy. It supports a centralized approach.	Does not consider node trustworthiness and draws more power. It only detects decreased rank attacks.	Cooja, Simulator	✓	×
[10]	Anomaly-based IDS and	ETX based, Detection	Introduces an anomaly IDS to detect malicious packet dropping, even under normal packet loss conditions. It supports a centralized approach.	Not focused on RPL-based networks. It is not designed to detect Rank attacks.	Cooja, Simulator	✓	×
[11]	Hybrid IDS	SVELTE	Detects routing attacks such as spoofed/ altered information, sinkholes, and selective forwarding; successful in identifying all malicious nodes in RPL networks. It supports a centralized and distributed approach.	Higher false positive rate; vulnerable to coordinated attacks. Rank attacks are not explicitly addressed.	Cooja, Simulator	✓	×
[12]	Cryptography-based	LEADER	Detects both increased and decreased rank attacks; employs distributed modules on nodes and a centralized sink module using modified DAO messages with added info. It supports a centralized and distributed approach.	Energy-intensive due to HMAC-LOCHA computations. It increases computational complexity.	Cooja Simulator	✓	×
[34]	Trust-based	S-MODEST	Uses a context-aware trust model and game theory (non-zero sum) to secure routes; maintains strong performance while defending against routing attacks.	Mobility and internal rank attackers are not considered.	Cooja Simulator	✓	×
[35]	Threshold-based	RTT	Detects and verifies decreased rank attacks using round-trip time (RTT); effectively removes malicious nodes and outperforms existing approaches. It supports a distributed approach.	Does not address increased rank attack scenarios	Cooja Simulator	✓	×
[36]	ML based	SVM	Proposes a centralized anomaly IDS using SVM; designed for smart hospital infrastructures; achieved high detection accuracy with low false positives. It supports a centralized approach.	Limited to single-anomaly detection. Does not account for increased rank attackers.	Cooja Simulator	✓	✓
[37]	Anomaly-based IDS	-	Detects rank and sinkhole attacks in multi-sink topologies using multipath routing; energy-efficient compared to single-path solutions. It supports a distributed approach.	Multipath design increases network overhead.	Cooja Simulator	✓	×
[38]	ML based	Deep Learning	Develops a deep learning-driven IDS to protect RPL networks against rank manipulation attacks, achieving high performance across accuracy, recall, and F1-score metrics. It supports a centralized approach.	Network efficiency metrics, including latency, energy consumption, and packet delivery ratio, are not evaluated.	Cooja Simulator	✓	×
[39]	ML based	MC-MLGBM	Enhances intrusion detection through optimized feature selection and a multiclass LightGBM-based classification framework.	Does not incorporate a broader range of adversarial scenarios, including additional RPL-specific and general sensor network attacks.	Cooja Simulator	✓	✓
[40]	ML based	ML based IDS	Introduces a ML-based detection framework combining genetic recursive feature selection with a fuzzy KNN classifier, achieving high predictive accuracy and a very low false positive rate. It supports a centralized approach.	Computationally intensive, as the genetic feature selection process is resource-heavy and unsuitable for highly constrained nodes.	Cooja Simulator	✓	×

standardized by the Internet Engineering Task Force (IETF) for 6LoWPAN, a key technology for IoT [41]. RPL is specifically designed for resource-constrained IoT environments. It organizes nodes into Destination-Oriented Directed Acyclic Graphs (DODAGs), which are tree-like topologies built and maintained using control packets such as DODAG Information Object (DIO), Destination Advertisement Object (DAO), and DODAG Information Solicitation (DIS).

DODAG construction begins with the root node broadcasting DIO messages. Receiving nodes calculate their rank and per-hop routing metric and may respond with a DAO message to join the network. Upon verification, the root node creates a routing table entry and confirms participation with a DAO-ACK. A node can only become a DODAG member and transmit packets after receiving this acknowledgment. Without DAO messages, DAO-ACKs are not sent, preventing node membership for transmission. If no DIO messages are received for an extended period, a node may issue a DIS message to solicit DODAG information. Fig. 2 illustrates the control message flow for DODAG formation and maintenance.

RPL networks are structured hierarchically with a designated root node. Other nodes join this structure based on their relative distance from the root. The protocol uses a metric called Rank to define each node's position within the network. This Rank value is crucial for routing decisions: it increases moving downward in the tree and decreases moving upward, as shown in Fig. 3. Because routing decisions rely on the Rank value, RPL is particularly vulnerable to Rank attacks, where malicious nodes exploit this metric to introduce false routing information.

While RPL is also susceptible to other threats, including selective forwarding, DAO inconsistencies, DIO/DIS suppression or flooding, energy-depletion, Sybil, and worst-parent attacks, these are not the focus of this study. The primary reason for concentrating on Rank attacks is that they directly manipulate the core routing metric, potentially affecting the entire DODAG structure and network stability. Addressing Rank attacks allows for a focused evaluation of mitigation strategies and provides a basis for future research on hybrid security mechanisms against other RPL attacks.

3.2. RPL rank attacks

RPL protocol assumes that all nodes within the DODAG are trustworthy and adhere to protocol rules. However, it lacks built-in security mechanisms to monitor node behavior [30], making it vulnerable to various attacks, broadly categorized into three categories [41]:

- Resource-exhaustion attacks (DoS attacks): Exploit limited network resources such as energy, memory, or bandwidth, reducing network lifespan and reliability.
- Topology- or routing attacks: Manipulate network structure, creating loops, suboptimal routes, or isolating nodes. Rank attacks fall into this category.

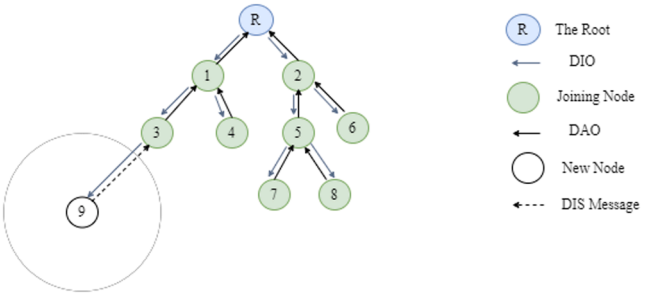


Fig. 2. Formation of DODAG by using different control messages.

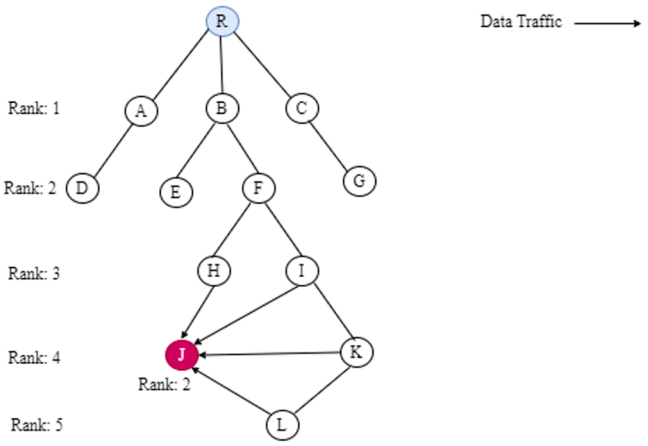


Fig. 3. Illustration of Rank attack in RPL network.

- Traffic-targeted attacks (Information disclosure): Eavesdrop or analyze traffic to gain intelligence that may facilitate attacks from the other two categories.

In RPL, source nodes select their preferred parent based on a lower rank value, which is fundamental for establishing efficient, loop-free routing paths. Standard objective functions, such as OF0 and MRHOF, favor nodes with lower ranks as next hops toward the DODAG root.

3.2.1. Decreased rank attack

In a decreased rank attack, a malicious node falsifies its rank value to appear closer to the root and broadcasts a DIO message with this fake rank. Neighboring nodes, perceiving it as a better parent, update their preferred parent to the attacker, routing traffic through it. This allows the attacker to compromise traffic and disrupt normal routing behavior by exploiting the rank metric.

3.2.2. Increased rank attack

In an increased rank attack, the attacker falsely advertises a higher rank. This manipulates neighbors or parent nodes into selecting alternative children, expanding the parent set artificially. Nodes compromised in this way may experience packet loss. Such attacks can disrupt data transmission, degrade network performance, form routing loops [42], reduce packet delivery ratio (PDR), and increase latency. RPL therefore requires standard security mechanisms to detect and mitigate these attacks, as discussed in later sections.

As shown in Fig. 3, node J acts maliciously by advertising a false rank of $R = 2$ while its actual rank is $R = 5$. Neighboring nodes H, I, K, and L select J as their preferred parent and begin routing packets through it. Initially, the parent-child relationships are: K is the parent of L, I is the parent of K, and F is the parent of H and I. After J advertises its false rank, nodes H, I, K, and L disconnect from their original paths and become compromised. Such scenarios are referred to as Rank Attacks or Rank Inconsistency Attacks.

Preliminary findings indicate that the RPL operating under Rank attacks exhibits compromised network performance. Key performance indicators reveal a packet delivery ratio (PDR) of 35%, along with an increase in control packet overhead and heightened latency attributable to excessive DAO message exchanges. This behavior also leads to a 75% increase in energy consumption, resulting from prolonged periods of node wakefulness necessary for these message transmissions. Furthermore, an evaluation of RPL under Rank attacks, utilizing various confusion metrics such as accuracy, precision, recall, and F_1-Score, consistently demonstrates suboptimal results across these measures.

4. RankShield: proposed framework

The RankShield framework is architected as a multi-tier security solution that bridges the gap between data-driven anomaly detection and deterministic threshold-based validation. The RankShield framework follows a hierarchical detection flow: first, a Deep Neural Network (DNN) acts as a high-sensitivity filter to identify statistical deviations in rank behavior. Second, a localized, threshold-based verification stage acts as a high-specificity validator to prevent the false labeling of legitimate nodes during network fluctuations. This hybrid approach ensures that the system remains resilient to the high-dimensional noise typical of congested IoT environments.

Key Techniques Employed

The framework utilizes three core techniques to ensure decentralized security:

- **Anomaly Classification:** A DNN model trained on multi-feature DIO datasets to distinguish between increased and decreased rank patterns.
- **Dynamic Thresholding:** A mathematical validation threshold logic that adapts to a node's depth (Hop Count) within the network, ensuring the threshold always corresponds to the current rank.
- **Blacklist Propagation:** A mitigation strategy that leverages standard RPL control messages (DAO) to isolate confirmed attackers without requiring additional specialized protocols.

The primary goal of this research is to detect and isolate both increased and decreased rank attackers within RPL-based IoT networks. While RPL is susceptible to various routing threats, RankShield is specifically designed to counter "Rank Inconsistency" attacks, where a malicious node falsifies its position in the DODAG to either attract traffic (decreased rank) or disrupt parent-child stability (increased rank). To achieve this, we propose RankShield, a hybrid detection and mitigation framework. The core idea behind RankShield is a multi-stage verification pipeline that balances high-accuracy deep learning with low-overhead threshold-based validation. The proposed RankShield framework ensures reliable and secure data transmission across heterogeneous nodes in large-scale, resource-constrained IoT deployments.

Integrating RankShield into the existing RPL protocol (RFC 6550) requires minimal yet strategic modifications to the control plane. Specifically, the integration involves a lightweight modification to the DIO processing function within the core RPL library (rpl-icmp6.c) to mirror incoming rank data to the RankShield module. Furthermore, the node's internal storage structure (rpl-private.h) is expanded to accommodate a 'Monitoring Table' for tracking neighbor rank history and blacklisted IDs. Finally, a verification step is inserted into the DAG management logic (rpl-dag.c) immediately after the Objective Function (OF) calculation, but prior to the official preferred parent update. These modifications are designed to be 'plug-and-play,' ensuring that RankShield can be implemented on standard IoT motes without requiring a complete overhaul of the RPL message structure or objective functions. In the following subsections, we provide a detailed explanation of RankShield's operational principles, including detection, verification, and mitigation strategies.

4.1. DNN-based rank attack detection mechanism

The first stage focuses on identifying rank attacks using a DNN model specifically designed for RPL networks. The DNN acts as a first line of defense by detecting rank attack events in real time and classifying them

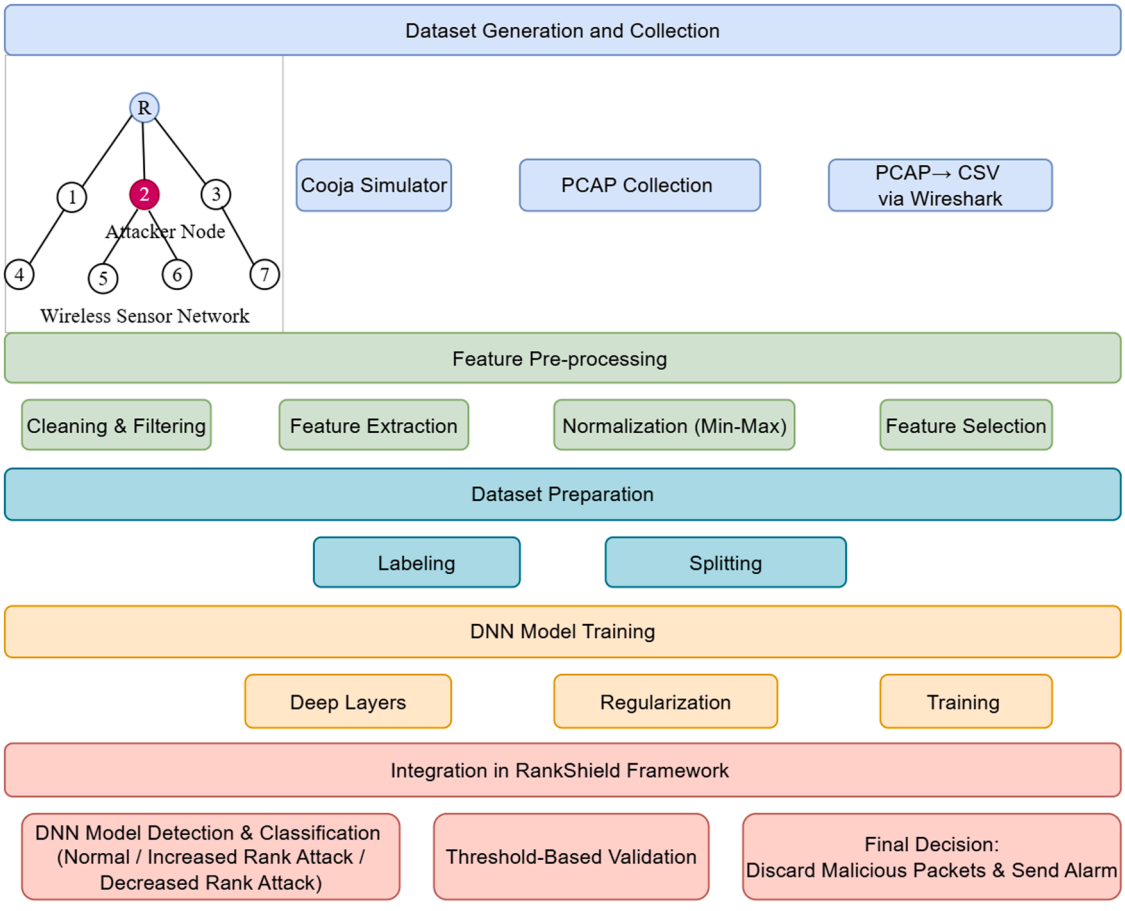


Fig. 4. Proposed DNN model for rank attack detection mechanism in RankShield framework.

into increased or decreased rank attacks, providing detailed insights into the nature of the threat.

The proposed DNN-based rank attack detection mechanism aims to act as a robust first line of defense by discriminating between normal and malicious rank values. As illustrated in Fig. 4, the methodology of the developed detection mechanism starts with data generation phase to gathers data on both malicious and benign node behavior by simulating rank attacks and normal RPL operation. This step involved collecting relevant data from the RPL network to analyze changes in DIO fields, particularly rank value. We specifically gathered instances of abnormal increases or decreases in rank value. Following continuous data collection, phase 2 includes relevant features are pre-processed, cleaned and extracted. Next, in the detection phase, these extracted features (i.e., aggregated statistics of DIO messages) are fed into a trained DNNs model. Specifically, DNNs were selected to identify rank values that deviate significantly from the normal rank. The process of generating datasets (i.e., consisting of normal RPL operation and RPL under the rank attack) is executed using the Cooja open-source simulator [43], with PCAP used to convert the analyzed data into a packet capture (PCAP) file. The PCAP file was then converted into a CSV file using Wireshark. This CSV file served as the source for extracting, filtering, and converting the data into a readable format. Our system ingests the CSV files into its feature pre-processing module. The features themselves are derived from the traffic flow data contained within these CSVs. This process begins with feature extraction, followed by data normalization applied uniformly across all datasets to mitigate the adverse effects of extreme values. Subsequently, a pre-feature selection phase involves dropping certain features based on a thorough feature importance analysis. The data was checked and pre-processed using Python's NumPy and Pandas libraries. After feature pre-processing, we label the datasets for each scenario and split them into training and mixed sets. This creates a pre-processed dataset containing a blend of both attack and benign data. These datasets then feed into DNN model. The deep layers of the DNN are trained using regularization and dropout mechanisms to adjust their weights. The detailed process is discussed as follows.

4.1.1. DNN model

The DNN employed in this research is a multi-layered architecture designed to automatically learn complex, non-linear relationships between input features. Beyond detection, the model can classify attacks into increased or decreased rank attacks, providing granular insights into the nature of each threat. The deep architecture of the DNN is particularly advantageous for this task because rank attack patterns often involve subtle, high-dimensional, and non-linear feature interactions that traditional ML models (e.g., Logistic Regression, Decision Trees, or SVMs) may fail to capture. Additionally, unlike classical ML models that often require extensive feature engineering, the DNN can perform end-to-end learning from raw or preprocessed features, reducing human bias and improving detection accuracy.

Training a DNN model involves a process called backpropagation, where the network's weights and biases are iteratively adjusted to minimize the difference between its predictions and the actual target values. This optimization is typically achieved using gradient descent or its variants, which calculate the gradient of the loss function with respect to the network's parameters. To prevent overfitting and enhance generalization to unseen network traffic, the model incorporates regularization and dropout mechanisms during training. This ensures that the DNN remains effective in real-time scenarios, detecting and classifying rank attacks as they occur within the network.

The model is trained on a comprehensively collected dataset that captures a wide variety of typical IoT traffic behaviors, including varying load conditions, and network dynamics. We utilize feature preprocessing and feature section techniques that extract relative and statistical metrics (e.g., rank variation, packet rate changes, inter-arrival time distributions), making them more resilient to shifts in overall network volume or composition. Furthermore, the DNN used in this research is designed with a periodic retraining capability. This allows the model to adapt to evolving network dynamics and emerging traffic patterns by incorporating the latest network data, ensuring its continued relevance and effectiveness over time.

We implement a robust data pre-processing and normalization step that includes noise reduction filters and outlier detection mechanisms. Initially, all attributes are normalized using the min-max normalization technique (discussed in Sub-Section 4.1.3). This step is crucial for cleaning raw sensor or network data, mitigating the impact of erroneous readings or irrelevant background traffic before it reaches the DNN.

The features selected for our model were chosen for their inherent robustness to common network noise. Regularization techniques such as L1 regularization are applied, which can drive the weights of less important features to be exactly zero. Moreover, the application of dropout layers (discussed in Sub-Section 4.1.4) specifically helps the model learn more robust features by preventing over-reliance on any single input feature that might be susceptible to noise.

4.1.2. Dataset generation

The initial phase involves obtaining a dataset with sufficient amount of samples. This dataset is generated by capturing messages transmitted between nodes in an RPL network through a Cooja simulation. The simulation includes both benign nodes and malicious nodes executing a rank attack, where information from DIO messages, specifically rank values, is collected.

The dataset is generated using PCAP files from Contiki-based nodes within the Cooja simulator and has been validated with 2000 wireless Zolertia nodes, confirming its suitability for developing a scalable attack detection model for an IoT network. These nodes were used only for validation purposes. For the protocol performance analysis, we used 40 and 100 nodes for the random network topology and 19 nodes for the restricted network topology. The raw data (as shown in Table 3), captured in a PCAP file, is converted to a CSV file using Wireshark and then labeled to distinguish between "attack" and "benign" data. For training purposes, the dataset is partitioned into three batches, with 70% of each batch used for training and the remaining 30% for testing. Table 3 shows the dataset recorded during RPL network simulation. This dataset is similar to the dataset used in the IoT Routing Attack paper [44], with the addition of a new dataset of attack-free data as well as increased rank attacks.

4.1.3. Feature pre-processing and normalization

In the second phase, feature normalization is used as a standard preprocessing step to reduce dataset complexity and enable the DNN model to process data more appropriately than a raw dataset. Raw data files contain unnecessary information, such as packet length and source/destination node IP addresses, sensor errors, irrelevant background traffic that isn't malicious etc., which can introduce noise and lead to overfitting in the learning algorithm. To address this, the data is first cleaned and filtered using Python libraries like NumPy and Pandas. Since data captured from different network nodes have varying mean and variance due to the dynamic nature of the network topology, a feature normalization process is performed on all datasets to mitigate the impact of extreme values that could skew the model's training. Initially, all attributes are normalized using the min-max normalization technique [44], with its mathematical formulation provided in Eq. (4.1).

Table 3
Dataset Consisting of RPL-based rank attacks and Attack Free Data.

Attack (Malicious Node) Data/Attack Free Data	No of records
Decreased Rank Attack	49,873,385
Increased Rank Attack	49,873,385
Normal/ Benign	99, 900,523
Total	199,647,293

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (4.1)$$

Where,

X_{norm} : represents the normalized value.

X: the original feature value.

X_{min} : the minimum value of the feature in the dataset.

X_{max} : the maximum value of the feature in the dataset

4.1.4. Feature selection

In this phase, feature selection is crucial component of the data preparation. The process begins with feature extraction, where relevant features for the model are derived from the aggregated statistics of DIO messages within the CSV files. After extraction, a feature importance analysis is conducted as a pre-selection step to drop less important features and prevent overfitting. This involves carefully removing the least significant features, while ensuring that the process does not lead to under-fitting.

To further enhance generalization, regularization techniques such as L1 regularization are applied, which can drive the weights of less important features to be exactly zero. Additionally, dropout mechanism is used as a powerful regularization technique that randomly drops out a fraction of the neurons in each layer to prevent co-adaptation and encourage the learning of more robust and discriminative representations. Table 4 presents a list of the set of features (adapted from [44]) selected in the proposed dataset.

Algorithm 1 presents the pseudocode for the proposed DNN-based detection mechanism in RankShield framework, to identify and classify rank attacks. The DNN is used to capture complex non-linear relationships in the dataset, while the threshold-based validation ensures robustness by filtering abnormal rank variations that deviate significantly from normal behavior. Furthermore, the threshold value selects rank-related features while discarding irrelevant ones, as rank is crucial for distinguishing between normal and malicious nodes.

4.2. Threshold-based rank attack validation and mitigation

Once the DNN model flags suspicious rank behavior, a decentralized threshold-based validation process is applied to confirm or reject the detection. This additional step introduces a quantifiable benchmark for "normal" network behavior, reducing false positives and ensuring that only significant deviations are treated as attacks. The validation mechanism operates in two phases, as follows.

Table 4
Existing features in the dataset.

Features	Description
No.	Sequence No of the packet
Time Stamps	Simulation Time
Source Node ID	Source Node IP address
Destination Node ID	Destination Node IP address
Length	Packet Length
Info	Packet Information
TR	Rate of Transmission
RR	Rate of Reception
HC	Hop Count
TAT	Transmission Average Time
TPC	Transmitted Packets
TRT	Total Reception Time
RPC	Received Packets
RAT	Reception Average Time
TTT	Total Transmission Time
DIS	DIS Packets
DIO	DIO Packets
DAO	DAO Packets
Label	Class Benign/Malicious

4.2.1. Inconsistency-checking with dynamic thresholds

The first phase of validation relies on a comparative analysis between observed rank behavior and the historical checking table. In this phase, each node initializes a checking table to maintain records of neighbor IDs and their corresponding rank values. When a node receives a DIO message from a new neighbor, it evaluates whether the advertised rank is acceptable that is, not excessively higher than its own rank. If the condition is satisfied, the neighbor's ID and rank are added to the existing routing table; otherwise, the DIO message is disregarded. For neighbors already present in the table, the node compares the newly received rank with the stored value. If the difference falls within a threshold, the entry is updated accordingly. However, if the rank deviates significantly beyond this threshold, the neighbor is marked as suspicious and further action is triggered. The RankShield dynamically and instantly updates its threshold value. This adjustment reflects changes in a neighbor's position within the network, ensuring the threshold always corresponds to the current rank. To satisfy the requirement that the threshold always corresponds to the current rank and reflects changes in a neighbor's position, the threshold (T) for a neighbor (j) as perceived by node (i) is typically defined in Eq. (1) as follows:

$$T = \left\lfloor \frac{R_{j(stored)}}{\text{MinHopRankIncrease}} \right\rfloor \times k \quad (1)$$

Where:

T: The calculated dynamic threshold value.

$R_{j(stored)}$: The last known valid Rank of neighbour j recorded in the checking table.

Min Hop Rank Increase: The standard RPL parameter (usually 256) defining the minimum increase in Rank between a child and its parent.

k: A sensitivity coefficient (or weighting factor) defined by the network to allow for minor fluctuations in link quality or Expected Transmission Count (ETX). In a strict RankShield implementation, k is often set to 3.

Therefore, considering varying node's position from traffic source (Hop Count) in the DODAG and the values of the $R_{j(stored)} = 256$ (Hop1), 512 (Hop 2), 768 (Hop 3) and 1024 (Hop 4), the given computed threshold, T is equal to:

$$T = 3,\ 6,\ 9,\ 12 \quad (2)$$

The threshold used in this research is not a static constant for the whole network. Because it is a function of $R_{j(stored)}$, it moves with the neighbor. If a neighbor legitimately moves further away from the root node or traffic source and successfully updates its rank within the allowed step-size, the threshold for the *next* update resets based on that new value. Unlike static security schemes, this threshold is topology-aware. Meaning, as a neighbor moves further from the root (increasing its Hop Count), the allowed variance grows proportionally, reflecting the natural accumulation of ETX fluctuations in deeper layers of the DODAG.

4.2.2. Mitigation through alerts and blacklisting

In the second phase, when a deviation beyond the threshold is confirmed, the receiving node designates the sender as malicious. It then broadcasts an alert message containing the malicious node's ID through DAO messages, which propagate downstream toward child nodes. The malicious node's ID is added to the blacklist, and any future traffic from that node is discarded. Because RPL networks typically maintain multiple parent options for routing, excluding suspicious nodes does not disrupt network performance, thereby preserving efficiency while mitigating threats. This decentralized threshold-based mechanism enhances the robustness of the DNN model by validating its predictions and preventing unnecessary alarms. Together, the DNN classifier and threshold-based validation form a layered defense within the RankShield framework, enabling accurate detection, classification, and

Algorithm 1
Proposed DNN-based rank attack detection in RankShield.

Input: Collect relevant data (*DIO, Rank*)
Output: Perform (min-max_norm)
D_train = {(features_i, label_i)} // Training Dataset and label_i ∈ {Normal, Attack}
Detect_Rank (T_rank_change) // Function call
i = 1;
Feature_List = ();
Threshold_Value == T_rank_change; // Analyze Threshold for Rank Change
For features *n* // features from the given dataset
Do
while (i < *n*) // Select two parent set features and apply cross validation
For (Feature(i)) // Evaluate Function value
If (Function_vlaue(Feature(i) == Threshold_value)
Feature_list = Feature(i)
else
Drop Feature_list
i = i++;
end
For features *x1, x2, …xn* // For all the features x1, x2, …xn
do
R ← *x1, x2, …xn* // Use the ranking and perform recursive feature selection from attributes
End
return (N_{Normal}, N_{Attack})
End

mitigation of increased or decreased rank attacks in RPL networks. Fig. 5 presents a flowchart of the rank attack validation and mitigation operation of the RankShield. The process begins upon the arrival of a DIO control packet. The framework immediately subjects the packet to a "Boundary Check." If the advertised Rank exceeds the calculated threshold relative to the node's current position, the transition follows the suspicious path: the neighbor is flagged, the DIO is dropped, and the node is added to a localized blacklist. If the deviation is confirmed as malicious, the mitigation logic triggers a "Network-Wide Awareness" event. The node generates an alert, embedded within a modified DAO message, which propagates downstream. This ensures that child nodes, which may not have directly witnessed the attack are pre-emptively warned to avoid the malicious parent. This decentralized isolation preserves the network's PDR by rerouting traffic through legitimate secondary parents already maintained within the RPL neighbor set.

Algorithm 2 systematically details the steps involved in validating and subsequently neutralizing a rank attacker (i.e., increased and decreased rank attacks) within the RankShield algorithm.

4.3. Theoretical analysis of congestion-aware threshold selection

A critical challenge in RankShield is distinguishing between malicious rank manipulation and legitimate rank fluctuations caused by network congestion. In high-density IoT deployments, "noise" in the form of packet collisions and MAC-layer delays can lead to transient rank inconsistencies [45]. To ensure the framework remains robust against this high-dimensional adversarial noise, RankShield employs congestion aware, adaptive thresholding logic, which is discussed in detail as follows.

4.3.1. Stochastic modeling of rank variance

We model the advertised rank of a node i at t time as $R_i(t)$. In a stable network, the change in rank ΔR between successive DODAG Information Object (DIO) messages should follow the constraints defined by the Objective Function (OF). However, under congestion, the Expected Transmission Count (ETX) fluctuates. We define the rank variance σ^2 as:

$$\sigma^2 = E\left[\left(R_i(t) - \mu_R\right)^2\right] \tag{3}$$

Where μ_R is the moving average of the rank. RankShield's threshold T is not static; it is defined as a function of the current network Traffic Load (γ) and the historical variance:

$$T_{dynamic} = T_{base} + \alpha \cdot \ln(1+\gamma) + \beta \cdot \sigma \tag{4}$$

Here, α and β are sensitivity coefficients. T_{base} represents the Base Threshold of the RankShield system. It is the starting sensitivity level of the detection mechanism under ideal or stable network conditions (when traffic load and variance are at a minimum). The logarithmic term $\ln(1+\gamma)$ ensures that as congestion increases, the threshold expands non-linearly, providing a buffer that prevents the system from misclassifying legitimate congestion-induced rank shifts as attacks.

4.3.2. Avoiding false positives via temporal correlation

To prevent the inadvertent exclusion of legitimate nodes during peak traffic, the inconsistency-checking mechanism applies a Temporal Validation Window (W). Instead of blacklisting a node based on a single threshold breach, RankShield evaluates the persistence of the anomaly:

$$P(attack) = \frac{1}{W} + \sum_{k=t-W}^{t} 1\left(\left|R_{oserved} - R_{predicted}\right|\right) > T_{dynamic} \tag{5}$$

Where:

$R_{oserved}$: This is the real-time rank value currently being advertised by a node in the network.

$R_{predicted}$: This is the expected rank value generated by the decentralized DNN model. It represents what the rank should be, calculated based on the multi-feature input vector (including DIO Emission Frequency, Neighbor Count, and Trickle timer intervals)

A node is only flagged as malicious if the probability $P(attack)$ exceeds a confidence interval (i.e., 0.8). During periods of high congestion, $T_{dynamic}$ increases and the observation window W is temporarily extended. This ensures that transient noise which is typically high-dimensional but short-lived does not trigger the mitigation protocol, whereas the systematic, persistent deviations characteristic of Rank Attacks are accurately captured.

The decentralized DNN model addresses adversarial noise by utilizing a multi-feature input vector that includes not only rank but also DIO Emission Frequency and Neighbor Count. By cross-referencing rank changes with the trickle timer interval, the DNN can identify when a node is "spamming" rank updates to destabilize the topology, a behavior distinct from the natural back-off patterns seen during legitimate network congestion.

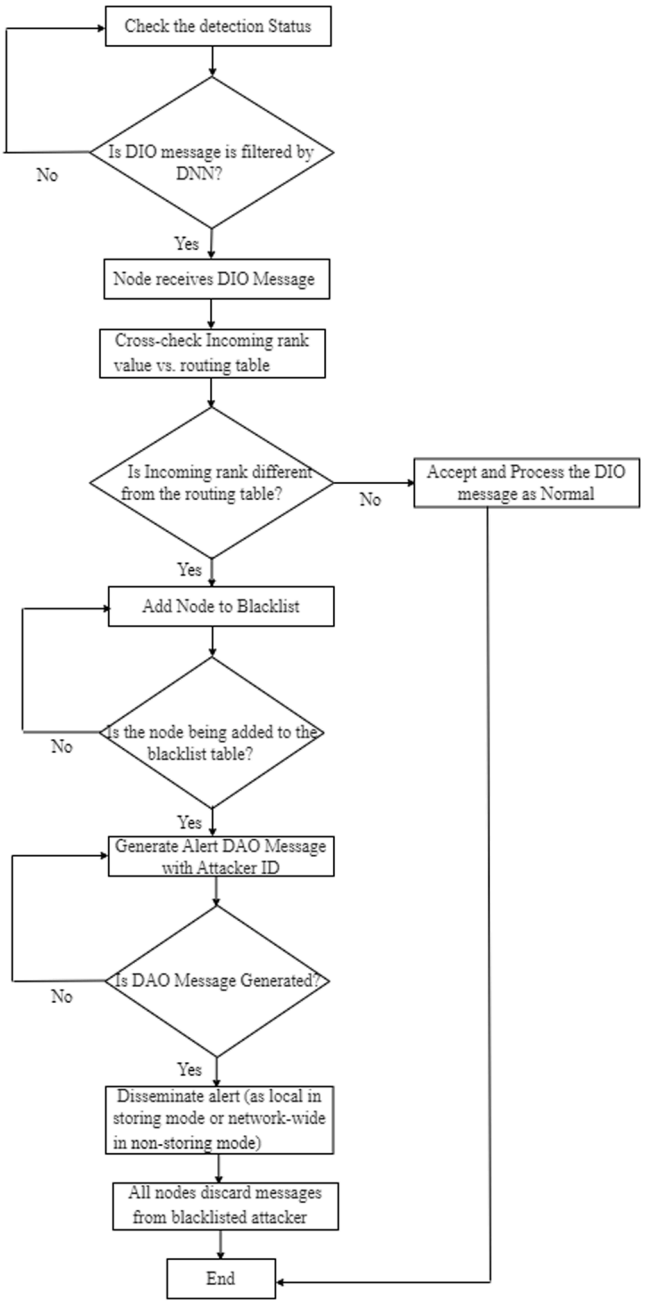


Fig. 5. Rank attack verification and mitigation operation of the RankShield protocol.

5. Simulation and results analysis

This section describes the simulation environment, the performance parameters employed to evaluate the proposed RankShield, and a discussion of the findings.

5.1. Simulation settings

To evaluate the impact of increased and decreased rank attacks on network efficiency and to test the effectiveness of the proposed RankShield, we conducted a series of experiments using Contiki OS, a lightweight and open-source operating system specifically designed for resource-constrained IoT devices [41]. To ensure accurate emulation of the binary code running on actual sensors, we utilized Cooja [46], a cross-level simulator for Contiki OS. Our simulations within Cooja employed MSPsim [46], an internal hardware emulator, to replicate the hardware limitations of the Zolertia Z1 platform, an MSP430-based board equipped with an ultra-low power IEEE 802.15.4 CC2420 radio transceiver.

For our network configuration, we used the Unit Disk Graph Radio Medium (UDGM) radio protocol, the CSMA/CA protocol at the MAC layer, and TSCH as the radio duty cycling (RDC) protocol. The RPL stack files were modified to accommodate the necessary data structures, and our proposed work was automatically triggered by the nodes upon network initialization. At the application layer, we simulated a periodic data collection scenario in which each node transmitted one packet per second to the sink. Upon completion of each simulation, the traffic was captured in a PCAP file, which was later converted into a CSV format using Wireshark for further analysis. The specific experimentation parameters used in our simulations are summarized in Table 5.

Three deployment scenarios were considered: a restricted topology, a random topology, and a random dense topology each spanning a 100 m × 100 m area. All nodes were static, with a transmission and interference range of 50 m. Each scenario included the root node (Node 1), ten malicious nodes (five executing increased rank attacks and five executing decreased rank attacks), and the remaining normal nodes (40 in the random topology, 100 in the random dense topology and 19 in the restricted topology). The root node was positioned at the center of all topologies. Each simulation was executed and repeated ten times for all topologies to ensure consistency and statistical reliability. The resulting network topologies are illustrated in Fig. 6.

5.1.1. Adversarial modeling and attack implementation

To evaluate the framework's resilience, we simulated adversarial behavior by intentionally compromising the integrity of the routing hierarchy. The Contiki-NG RPL library was extended to implement both decreased and increased rank attacks. The core mechanism of these attacks involves the manipulation of the control plane to broadcast deceptive topological information. Specifically, we modified the 'rpl-icmp6.c' and 'rpl-dag.c' core library files within the Contiki environment. By altering the Rank-advertisement parameters, we enabled malicious nodes to bypass the standard Objective Function (OF) constraints.

These modifications allow the simulated attacker to inject false DIO messages directly into the network stream, providing a realistic setup for the RankShield detection module. RankShield identifies these simulated attacks through a continuous monitoring of the ICMPv6 control traffic. Detection is triggered the moment a neighbor's advertised Rank deviates from the expected statistical and mathematical bounds calculated by the dynamic threshold logic in Section 4. When a malicious DIO is intercepted, the system records the discrepancy in the local Monitoring Table. If the inconsistency is confirmed, the framework immediately drops the deceptive packet and initiates the blacklist propagation process described in Section 4. This ensures that the attack is neutralized at the ingest point before it can corrupt the node's routing table or be passed on to child nodes.

5.2. Network performance metrics

The network performance metrics considered for the evaluation of the RankShield framework are:

1. Average PDR: defined as the ratio of the total data packets received by the destination node to the total data packets sent by all RPL nodes in the network.

$$\text{PDR} = \frac{\textit{Total data packets received by the destination node}}{\textit{Total data packets sent by the RPL node}} * 100$$

Algorithm 2
Rank attack validation and mitigation mechanism.

```
// Function to initialize the checking table
void initialize_checking_table() {
  for (int i = 0; i < MAX_NEIGHBORS; i++) {
    checking_table[i].exists = false;
  }
}
// Function to initialize the blacklist table
void initialize_blacklist() {
  for (int i = 0; i < MAX_BLACKLIST; i++) {
    blacklist_table[i].blacklisted = false;
    strcpy(blacklist_table[i].node_id, "");
  }
}
// Function to check if a node is in the checking table
int find_neighbor_in_checking_table(const char *node_id) {
  for (int i = 0; i < neighbor_count; i++) {
    if (checking_table[i].exists && strcmp(checking_table[i].node_id, node_id) == 0) {
      return i; // Found
    }
  }
  return -1; // Not found
}

// Function to add a node to the checking table
void add_to_checking_table(const char *node_id, int rank) {
  if (neighbor_count < MAX_NEIGHBORS) {
    strcpy(checking_table[neighbor_count].node_id, node_id);
    checking_table[neighbor_count].rank = rank;
    checking_table[neighbor_count].exists = true;
    neighbor_count++;
    printf("Added neighbor %s with rank %d to the checking table.\n", node_id, rank);
  } else {
    printf("Checking table is full. Cannot add neighbor %s.\n", node_id);
  }
}

// Function to update a node's rank in the checking table
void update_checking_table(int index, int new_rank) {
  checking_table[index].rank = new_rank;
  printf("Updated rank of neighbor %s to %d in the checking table.\n", checking_table[index].node_id, new_rank);
}

// Function to process a received DIO message
void process_dio(const char *sender_id, int sender_rank) {
  printf("Received DIO from node %s with rank %d.\n", sender_id, sender_rank);
  if (is_blacklisted(sender_id)) {
    printf("Node %s is blacklisted. Ignoring DIO.\n", sender_id);
    return;
  }
  int existing_neighbor_index = find_neighbor_in_checking_table(sender_id);
  if (existing_neighbor_index == -1) {
    // Sender not in the checking table
    if (sender_rank ≤ current_node_rank + 5) {
      add_to_checking_table(sender_id, sender_rank);
    } else {
      printf("New neighbor %s has a significantly higher rank (%d). Ignoring.\n", sender_id, sender_rank);
  }
  } else {
    // Sender exists in the checking table
    if (checking_table[existing_neighbor_index].rank == sender_rank) {
      printf("Rank of existing neighbor %s is consistent (%d).\n", sender_id, sender_rank);
      // Maintain the entry (no action needed)
  } else {
      printf("Rank inconsistency detected for node %s. Old rank: %d, New rank: %d.\n",
      sender_id, checking_table[existing_neighbor_index].rank, sender_rank);
      add_to_blacklist(sender_id);
      // Optionally, remove from checking table:
      checking_table[existing_neighbor_index].exists = false;}
  }}
// Function to check if a node is in the blacklist
bool is_blacklisted(const char *node_id) {
  for (int i = 0; i < blacklist_count; i++) {
    if (blacklist_table[i].blacklisted && strcmp(blacklist_table[i].node_id, node_id) == 0) {
      return true;
    }
  }
```

(*continued on next page*)

Algorithm 2 (*continued*)

```
    return false;
}
// Function to add a node to the blacklist
void add_to_blacklist(const char *node_id) {
    if (!is_blacklisted(node_id) && blacklist_count < MAX_BLACKLIST) {
        strcpy(blacklist_table[blacklist_count].node_id, node_id);
        blacklist_table[blacklist_count].blacklisted = true;
        blacklist_count++;
        printf("Node %s added to the blacklist.\n", node_id);
        // Simulate broadcasting alert message via DAO
        printf("Broadcasting alert DAO about malicious node %s.\n", node_id);
    } else if (is_blacklisted(node_id)) {
        printf("Node %s is already in the blacklist.\n", node_id);
    } else {
        printf("Blacklist is full. Cannot add node %s.\n", node_id);
    }
}
int main() {
    // Set the current node's rank
    current_node_rank = 10;
    printf("Current node rank: %d\n", current_node_rank);
    // Initialize tables
    initialize_checking_table();
    initialize_blacklist();
    printf("\nChecking Table:\n");
    for (int i = 0; i < MAX_NEIGHBORS; i++) {
        if (checking_table[i].exists) {
            printf("ID: %s, Rank: %d\n", checking_table[i].node_id, checking_table[i].rank);
        }
    }
    printf("\nBlacklist Table:\n");
    for (int i = 0; i < MAX_BLACKLIST; i++) {
        if (blacklist_table[i].blacklisted) {
            printf("Blacklisted Node: %s\n", blacklist_table[i].node_id);
        }
    }
    return 0;
}
```

2. Control message overhead: the total count of control packets exchanged within the RPL network.

$$\textit{Control Message Overhead} = \textit{Total count of control packets within the RPL network}$$

3. Average delay for the transmission of data packets: the average time it takes for data packets to be successfully transmitted and received by the sink node, not including packets that were dropped or lost.

$$\textit{Average Delay} = \sum_{i=1}^{N} \frac{\textit{Delay}_i}{\mathrm{N}}$$

where:

- $\textit{Delay}_i$: is the end-to-end delay for the i th successfully received data packet.
- N: is the total number of data packets successfully received by the sink node.

1. Average energy consumption: is the total energy used by a node for processing, sending, and receiving packets during a simulation. It is determined by multiplying the time a node is active by its power consumption, then summing these values. The time a device spends in different states, such as low-power mode, CPU processing, transmitting, or receiving packets, is tracked using "Energest variables," which measure the duration in clock ticks. The total energy is the sum of these products, while the average power is that sum divided by the total simulation time. The power required by a mote's components is calculated by multiplying the current in each operating mode by the supply voltage.

$$\textit{Average Energy Consumption} = \sum_{s \in S} E_s$$

Where:

- S: is the set of all operating states (e.g., CPU active, radio transmitting, radio receiving, low-power mode).
- E_s: is the energy consumed in state s, calculated as:

$$E_s = P_s \times T_s$$

where:

- P_s: is the power consumed in state s ($P_s = V \times I_s$).

Table 5
Simulation configurations.

Parameters	Values
Simulator and OS	Cooja and Contiki-NG 5.0
Radio Medium	Unit Disk Graph Medium (UDGM)
Motes	Zolertia Z1
Area Size	200 × 200 m
Number of nodes	40 nodes + 1 RPL root (random topology), 100 nodes + 1 RPL root (dense random topology), 19 nodes + 1 RPL root (restricted topology)
Number of Malicious nodes	10 (5 increased rank attackers, 5 decreased rank attackers)
Topologies	2 Random and 1 Restricted topology
No of timeslots in Slotframe (SL)	17
Threshold (T)	3, 6, 9 and 12
Number of Iterations	10 for each configuration
Transmission and Interference Range	Tx: 50 m, Interference: 50m
Simulation duration	3 h and 45 min
RNG seeds	New seed for each iteration
Data Packet Size	40 bytes
Traffic Sending Rate (Normal Traffic Load)	1 pkt/s (100 packets sent by the sending node)
Bursty Traffic Load	5, 10, 15 and 20 Packets/Minute – 1000 packets sent by the sending node
Routing	RPL
MAC and PHY	TSCH with Orchestra scheduling, IEEE 804.15.4

- T_s: is the time spent in state s (measured in seconds).

The average energy consumption can be further simplified as follows

$$\begin{aligned} \textit{Average Energy Consumption} &= (V \times I_{CPU} \times T_{CPU}) + (V \times I_{TX} \times T_{TX}) \\ &+ (V \times I_{RX} \times T_{RX}) \\ &+ (V \times I_{LPM} \times T_{LPM}) \end{aligned}$$

Where:

- V: is the supply voltage (i.e., 3 V for a Zolertia mote).
- I_{CPU}, I_{TX}, I_{RX} and I_{LPM}: are the current draws for the CPU, radio transmitting, radio receiving, and low-power modes, respectively.
- T_{CPU}, T_{TX}, T_{RX} and T_{LPM} are the durations spent in each of these states. These durations are derived from the "Energest variables," which are measured in clock ticks and converted to seconds.

We used a confusion matrix to calculate key performance indicators. This matrix details the predicted classifications made by our DNNs. We evaluated performance using a test dataset. It is based on the counts of test data instances correctly and incorrectly predicted. The test dataset contains observed labels, which allow evaluation by comparing predicted result values to known result values. The confusion matrix is constructed from the four outcomes presented in Table 6.

If the model detects the attack data as an attack and that is correct, the classification is a true positive (TP), also called positive class accuracy or sensitivity. If the model does not detect malicious activity and it is true, this prediction is a true negative (TN), also called negative class accuracy or specificity. If the model indicates a malicious activity and it is erroneous, the classification is a false positive (FP), and if the model detects that there is no malicious activity but that it is false, the classification is a false negative (FN). High TP and TN (low FP and FN) prediction rates are acceptable and desirable. The performance evaluation metrics used to generate accuracy, precision, recall and F_1-Score are formalized as follows, using the confusion matrix.

2. Accuracy: indicates the proportion of correctly classified attack instances across both classes. A higher accuracy signifies a more effective model.

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)} * 100$$

3. Precision: it is the proportion of correct positive predictions to the total predicted positives. In simpler terms, it's the number of all correct instances divided by the total number of relevant and irrelevant attacks. It's also known as the Positive Predictive Value, with the best possible value of 1.0 (100%) and the worst of 0.0 (0%).

$$\text{Precision} = \frac{TP}{(TP + FP)} * 100$$

4. True Positive Rate or Recall: this metric is defined as the ratio of the number of correct positive predictions (true positives) to the total number of relevant attacks that truly occur in the network. It quantifies our detection's effectiveness by showing the ratio of correctly identified positive instances compared to the total number of real attacks. This metric is often referred to as the Detection Rate (DR).

$$\text{Recal} = \frac{TP}{(TP+FN)} * 100$$

5. F_1-Score: this metric presents a single, overall measure of a model's accuracy across the entire dataset. It combines recall and precision into one value using a weighted harmonic mean.

$$F_1 - \text{Score} = \frac{2 * \text{ Precision} * \text{ Recall}}{(\text{Precision} + \text{ Recall})} * 100$$

6. Detection Latency (DL): Detection Latency is the time delay between the moment a malicious DIO packet is first broadcast (T_{attack}) and the moment the RankShield framework ($T_{detection}$) successfully flags the node.

$$\text{DL} = T_{detection} - T_{attack}$$

5.3. Performance evaluation and discussion

This section presents an analysis of the proposed RankShield. To rigorously evaluate the performance of RankShield, we selected three distinct benchmarks that represent the current state of RPL routing and security. These selections begin with Standard RPL, evaluated both with and without attacks, to serve as the fundamental baseline for network performance. While the scenario without attacks provides the upper bound of efficiency under ideal conditions, the "RPL under attack" configuration demonstrates the maximum vulnerability and performance degradation, thereby establishing a clear gap for RankShield to bridge. Furthermore, SBIDS was chosen to compare RankShield against traditional, rule-based monitoring. As SBIDS represents a class of "static" defenders, comparing it against our framework highlights RankShield's specific advantage in utilizing dynamic thresholds that adapt to network congestion. Finally, we included ARM, a prominent hybrid IDS, to demonstrate how RankShield's decentralized architecture outperforms centralized models. This comparison is vital for highlighting improvements in energy efficiency and the elimination of single-point-of-failure risks inherent in centralized systems.

This comparison aims to measure the effectiveness and performance of our approach. Specifically, we assessed the performance based on average PDR, average delay, control packets overhead, global energy consumption, accuracy, precision, recall and F_1-Score.

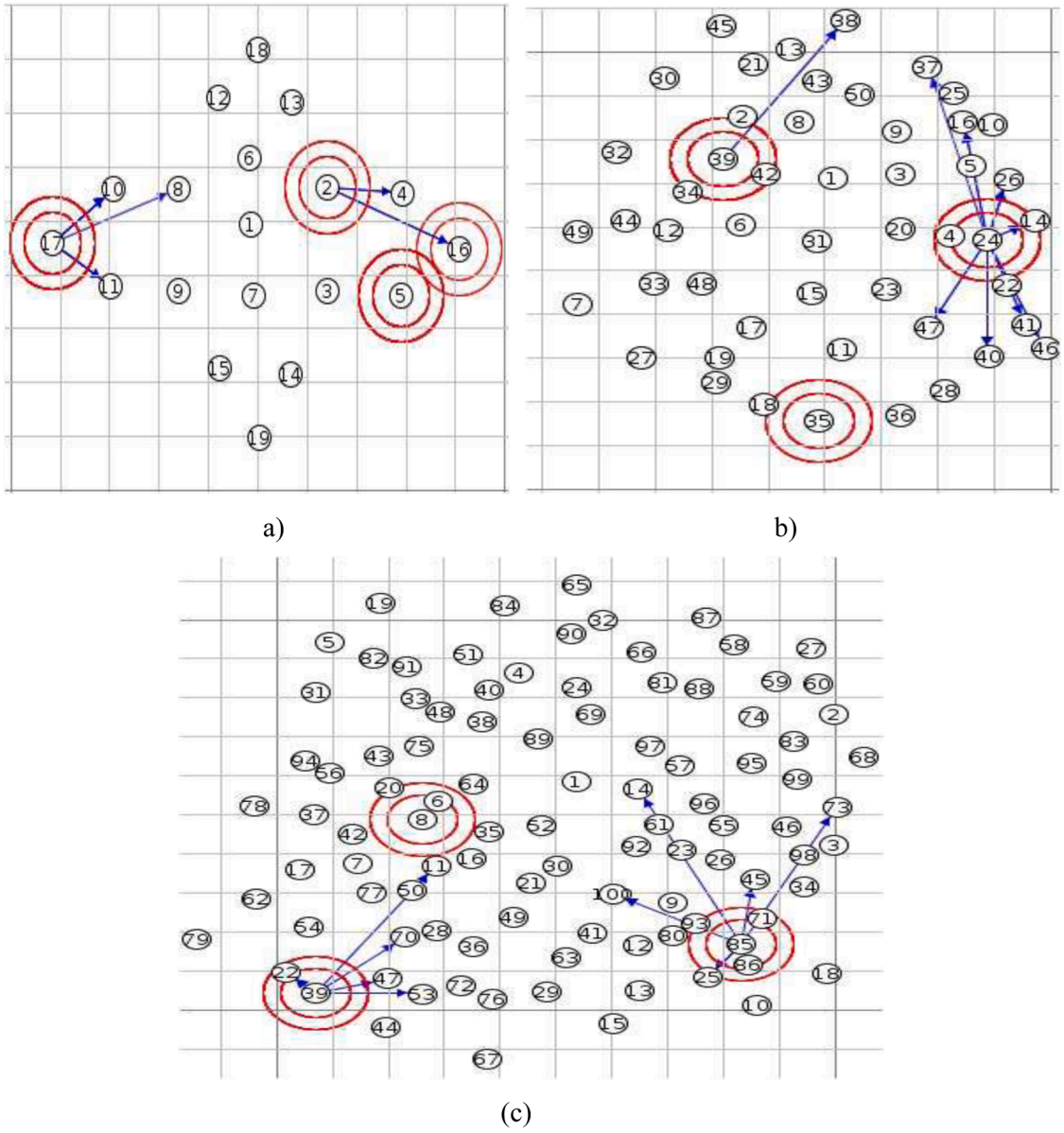


Fig. 6. Restricted (a) and random (b) random dense (c) network topologies.

5.3.1. Average PDR

Fig. 7 presents the network performance parameters, specifically the PDR, of Standard RPL (with and without attacks), SBIDS, ARM, and our proposed RankShield across both network topologies. As shown, RankShield achieved a higher PDR in the restricted topology compared to Standard RPL with Attacks, SBIDS, and ARM. This superior performance is attributed to the mitigation and isolation techniques employed by RankShield. Furthermore, in the random topology, RankShield also exhibited the highest PDR, outperforming SBIDS and ARM. This improvement is due to the refinement and validation mechanism used in RankShield. This mechanism acts as a critical second layer of validation. While the DNN identifies potential anomalies, the inconsistency-checking applies predefined or learned thresholds to specific network parameters or behaviors. This helps to confirm whether a detected anomaly truly constitutes an attack or is merely a benign fluctuation. Moreover, by cross-referencing DNN predictions with a robust set of consistency rules, RankShield can significantly reduce false positives (legitimate traffic flagged as an attack) and potentially improve recall by

Table 6
Confusion Matrix.

	Predicted	
Real (Actual)	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

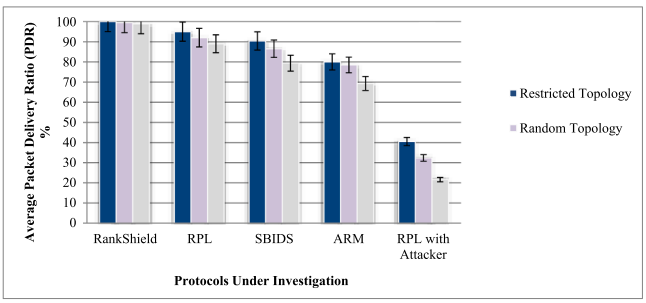


Fig. 7. Average packet delivery ratio, considering different network topologies.

catching edge cases that the DNN alone might not definitively classify. This hybrid approach leverages the strengths of both data-driven learning and rule-based validation.

Fig. 7 clearly shows that RankShield achieves a significantly better PDR compared to RPL, SBIDS, and ARM. For all algorithms except RankShield, the random topology consistently results in a lower PDR. This degradation in RPL (both with and without attacks), SBIDS, and ARM is primarily due to routing errors caused by both increased and decreased rank attacks, which lead to frequent DODAG reconstructions and subsequent packet drops. In stark contrast, RankShield maintains a perfect 100% PDR across both restricted and random network topologies. This is due to the use of the threshold-based system that provides a quantifiable benchmark for "normal" behavior. Deviations beyond this threshold, combined with DNN-identified patterns, lead to highly precise and robust attack identification.

The analysis of random dense network topologies with 100 nodes in Fig. 7 reveals that RankShield consistently maintains a 99.8% PDR, even as network complexity significantly increases. This performance surpasses Standard RPL (with and without attacks), SBIDS, and ARM, which experience considerable PDR degradation due to routing errors and frequent DODAG reconstructions in denser environments. RankShield's robust mitigation and isolation techniques, coupled with its hybrid DNN and consistency checking approach, effectively counteract the heightened risks of increased and decreased rank attacks in these complex settings. The threshold-based system provides a quantifiable benchmark for normal behavior, enabling highly precise and robust attack identification even in large-scale IoT networks.

5.3.2. *Control packets overhead*

Fig. 8 provides crucial insights into the efficiency of attack detection mechanisms by showcasing the control message overhead for RankShield compared to Standard RPL without Attacks, SBIDS, and ARM across both restricted and random network topologies. The results indicate that our proposed protocol, RankShield, demonstrated the best performance in these topologies by significantly reducing the number of redundant control messages produced compared to Standard RPL with Attacks, SBIDS, and ARM, which generated a higher volume of control messages. RankShield best performance in restricted topologies can be further explained by the nature of these environments. In a more predictable and structured topology, normal behavior is well-defined and less variable. This allows the DNN to learn highly precise baselines, and the threshold system to apply very stringent and accurate filtering criteria. The clarity of the environment means RankShield can detect and discard malicious messages with even greater certainty and speed, leading to minimal generation of redundant control messages in response to attacks.

The ability of RankShield to achieve the best performance in reducing control message overhead, is fundamentally linked to its integrated detection mechanism the DNNs combined with a threshold-based inconsistency-checking and malicious node removing mechanism. The DNNs embedded within RankShield are highly effective at learning intricate patterns characteristic of legitimate RPL routing behavior. Consequently, they can accurately identify even subtle deviations indicative of a rank attack almost instantaneously. This early and precise detection is crucial because it allows the protocol to identify and remove malicious control messages (e.g., fraudulent DIOs attempting to alter ranks) right at their inception. While the threshold-based inconsistency-checking mechanism acts as a critical validation layer. Once the DNN flags a potential attack, this mechanism rigorously verifies the anomaly against predefined or learned thresholds of "normal" behavior. When a rank attack is confirmed, RankShield's core strength lies in its ability to immediately discard these malicious control messages. This proactive discarding is the key to minimizing overhead. By preventing erroneous or malicious routing information from propagating through the network.

Fig. 8 clearly shows that the RankShield algorithm consistently generates fewer control packets than other mitigation methods across all network topologies. In fact, the number of control packets produced by RankShield is nearly identical to that of RPL with no attackers present. This efficiency is due to RankShield's design, which eliminates the need for extra control packets for rank attacker detection or alarm generation. Not to mention that other legitimate nodes are not tricked into reacting to false rank updates or perceived topology changes. This prevents the cascading effect of unnecessary control message exchanges (e.g., DIOs, and DAOs) that would normally be triggered by network instability or attempts to resolve non-existent routing loops. In contrast, Standard RPL with Attacks exhibit a higher volume of control messages, especially under attack conditions. Without any dedicated attack detection or mitigation, a standard RPL network under a rank attack becomes highly unstable. Malicious rank values or loops cause nodes to constantly try to update their routing tables, change parents, and re-establish paths. This leads to a flood of redundant control messages as nodes desperately attempt to restore connectivity, creating loops, retransmitting messages, and consuming immense network resources. This represents the high-overhead baseline that detection mechanisms aim to alleviate. While SBIDS and ARM do implement attack detection, their higher control message overhead compared to RankShield suggests that their mechanisms are not detect attacks as quickly or as accurately as RankShield's hybrid approach, allowing malicious messages to propagate further or persist for longer before mitigation. This prolonged presence of an attack leads to more "reactive" control message generation as nodes attempt to cope with the instability before the attack is fully neutralized.

In networks with 100 nodes, RankShield continues to demonstrate superior performance in managing control packet overhead, as illustrated in Fig. 8. Its integrated deep neural network (DNN) and threshold-based detection mechanisms are highly scalable. A larger number of nodes provides a more extensive dataset, enabling the DNN to enhance its understanding of normal network behavior. This capability for rapid and precise identification and rejection of malicious control messages becomes even more vital in dense network environments. In such settings, the risk of a cascading flood of redundant control messages originating from a single attack is considerably higher for less efficient protocols. Furthermore, when analyzing RankShield, there's a slight

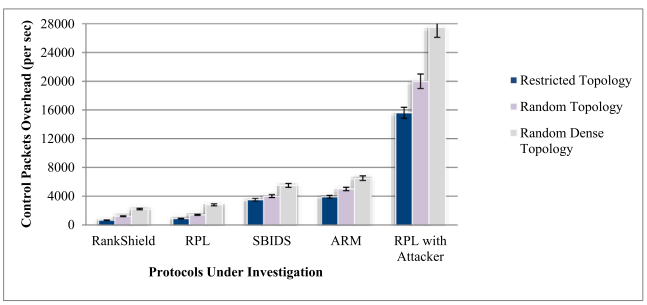


Fig. 8. Control packets overhead, considering different network topologies.

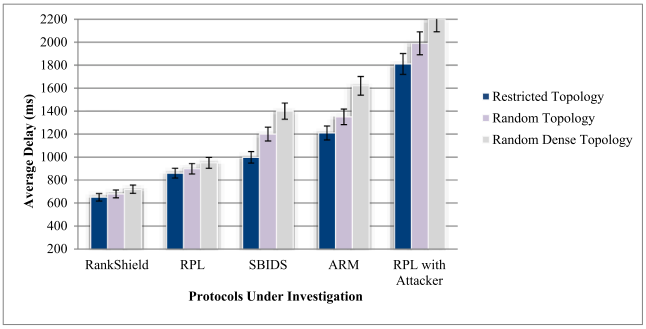


Fig. 9. Average delay, considering different network topologies.

increase in control packets within a random dense network topology compared to random topologies with 40 nodes. This difference is attributed to the increased propagation of DAO messages throughout the RPL network as the number of nodes grows. These DAO messages indicate the presence of an attacker, and their increased prevalence can lead to a rise in control traffic or packets, potentially impacting overall network performance.

5.3.3. Average delay

Fig. 9 shows that RankShield significantly reduces latency more efficiently than RPL with attackers, SBIDS, and ARM. RankShield 's exceptional ability to reduce average delay is a direct consequence of its sophisticated, two-step rank attack detection mechanism, which is the integration of DNNs with a threshold-based inconsistency-checking mechanism. This hybrid approach allows RankShield to mitigate the primary factors contributing to delay, particularly the generation of unnecessary control messages and the disruption of optimal communication paths. The core of RankShield's delay reduction lies in its capability to effectively filter malicious nodes by detecting and discarding rank attacks early. Once a malicious node or message is identified and filtered/discarded, RankShield prevents it from injecting erroneous rank information into the network. This is crucial because such attacks typically trigger a cascade of redundant control messages (e.g., DIOs, and DAOs) as legitimate nodes attempt to correct perceived routing inconsistencies, leading to frequent parent changes, routing loops, and network instability. By proactively stopping this control packet storm, RankShield directly alleviates channel congestion, reducing queuing and transmission delays for all packets.

RPL with Attacks represents the baseline scenario of an unmitigated attack. Rank attacks cause severe routing instability, leading to constant loops, and retransmissions. This inherently generates immense control overhead that clogs the channel and forces data packets onto convoluted or broken paths, resulting in the highest average delays as shown in Fig. 9. Fig. 9 also shows SBIDS and ARM protocols results a bit higher average delay compared to RankShield suggests several limitations. This is because their mitigation strategies cannot be as efficient at immediately preventing the generation of redundant control packets. They can detect the attack but struggle to quickly suppress its network-wide impact on routing message floods.

In random dense network topologies as shown in Fig. 9, RankShield's superior performance in reducing average delay becomes even more pronounced. The heightened node density in these scenarios typically exacerbates control message overhead and routing complexities, making efficient attack detection and mitigation paramount. RankShield's integrated DNN and threshold-based detection mechanism is particularly effective here, swiftly identifying and isolating malicious nodes before they can trigger widespread control packet storms across the larger network. This filtering prevents the rapid propagation of erroneous rank information, which would otherwise lead to extensive retransmissions and congestion in a dense environment. Consequently, RankShield maintains stable and optimized communication paths, significantly outperforming RPL with attackers, SBIDS, and ARM by preventing the exponential increase in delay that these protocols would experience under similar dense, attacked conditions.

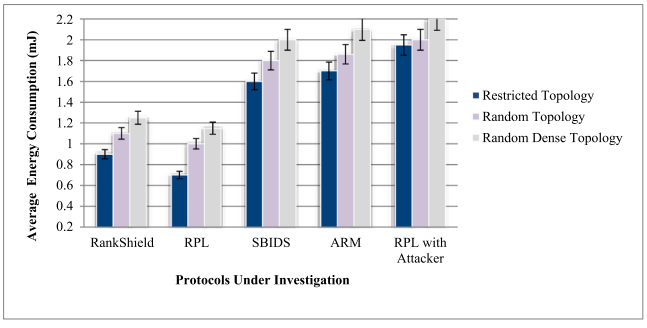


Fig. 10. Average energy consumption, considering different network topologies.

5.3.4. Average energy consumption

Fig. 10 presents the Average Energy Consumption of RankShield compared to Standard RPL with Attacks, SBIDS, and ARM across the two topologies (i.e., restricted and random). The results indicate that our proposed protocol, RankShield, significantly reduced the average energy consumption, achieving a reduction of up to 50% and exhibiting much lower energy consumption in the random topology than Standard RPL with Attacks, SBIDS, and ARM. RankShield's remarkable energy efficiency is directly attributed to its sophisticated, hybrid approach, which drastically reduces the factors that drive high energy consumption in RPL networks under attack. The hybrid approach which consists of DNNs within RankShield are highly effective at learning and identifying the subtle, non-linear patterns associated with rank attacks. This enables RankShield to detect malicious behavior (such as the injection of false DIO messages) very quickly and accurately. The threshold-based mechanism then rigorously validates these detections. Once a rank attack is confirmed, RankShield immediately prevents the propagation of erroneous or malicious control messages, especially DIOs. Unmitigated rank attacks cause a massive increase in DIO message exchanges as nodes constantly try to update their ranks and nodes in response to the attacker's manipulations. By effectively detecting and discarding these attacks, RankShield dramatically curbs the generation of redundant DIO messages. This reduction in DIOs, along with other associated control messages like DAOs, directly translates to lower overall control overhead. By significantly reducing the number of DIOs and other control messages, RankShield ensures that nodes do not need to be awake as frequently to exchange these messages in a given TSCH timeslot. This allows nodes to transition into low-power sleep states more often and for longer durations. This direct control over the "awake time" for message exchange is the primary reason for the up to 50% reduction in average energy consumption.

Fig. 10 clearly illustrates that random topologies consume more energy. This is because nodes in a random topology are often placed at relatively equal distances close to the root, leading to a higher density of nodes within each other's transmission range. The proximity results in a more frequent exchange of packets, including both data and control messages, as more neighbors are discovered and interacted with. This continuous radio activity significantly increases overall energy consumption. Additionally, the path diversity created by attackers further contributes to this elevated energy drain. Meaning, when attackers are present in random topologies, they can exploit the inherent diversity of paths. This can lead to more frequent route changes, re-routing attempts, and a general state of instability, all of which contribute to an elevated energy drain as nodes work harder to establish and maintain connectivity amidst the disruption.

Fig. 10 shows standard RPL with attacks, which represents the scenario of an unmitigated attack. Without any effective detection or defense, rank attacks cause severe and persistent network instability. Nodes are constantly engaged in re-routing efforts, frequent parent changes, and the continuous exchange of control messages (especially DIOs and DAOs) to adapt to the attacker's manipulations. This forces nodes to remain awake for prolonged periods, rapidly draining their limited battery power and resulting in the highest average energy consumption. While SBIDS and ARM protocols attempt to detect attacks, their higher energy consumption compared to RankShield suggests that their detection or mitigation strategies cannot be as rapid or effective at immediately suppressing the impact of malicious messages. This can lead to longer periods of network instability and a higher volume of control messages before the attack is fully addressed. Even after detection, they might not be as efficient at pruning unnecessary control traffic, leading to sustained energy drain from constant message

processing and transmission. Consequently, nodes using these protocols need to be awake more often, consuming more power than in RankShield's optimized environment.

For dense random network topologies as shown in Fig. 10, the energy consumption patterns observed in smaller random topologies would be a bit amplified. The increased node density within each other's transmission range would lead to a dramatic rise in packet exchanges, both for data and control messages, as more neighbors are discovered and interacted with more frequently. This continuous and intensified radio activity would result in substantially higher overall energy consumption compared to the 40-node random topology. RankShield's ability to reduce DIO messages and enable longer sleep states would be even more critical in such a dense environment, as it would prevent the exponential increase in energy drain that unmitigated rank attacks would cause due to rampant re-routing and control message overhead.

5.3.5. Accuracy

This paper analyzes decreased and increased rank attacks, comparing our results with RPL, SBIDS, ARM and RPL with attacker as shown in Fig. 11. In our analysis, the RankShield algorithm that uses the DNN model consistently demonstrates a higher accuracy in restricted network topologies compared to random topologies for rank attack detection. This observation aligns with the expected behavior, as the more predictable nature of restricted topologies often allows for more precise pattern recognition. As such the DNN can learn highly precise "normal" behavior profiles due to less variability. The threshold-based inconsistency-checking also becomes extremely effective as baselines are clearer and deviations are more pronounced, contributing to very high accuracy (as seen with RankShield's 99.8%). While random topologies inherently more dynamic and less predictable, the DNN's ability to learn complex, non-linear relationships shine through. It can adapt to the higher variability. The threshold-based mechanism then acts as a crucial filter, ensuring that flags are indeed indicative of attacks. This combination allows RankShield to maintain strong performance even where other protocols (like SBIDS and ARM, which seem to struggle more in random topologies) might falter due to reliance on simpler models or less adaptive heuristics.

Fig. 11 also illustrates that RankShield's accuracy is higher in comparison to that of RPL, SBIDS, and ARM. The superior performance of the proposed RankShield rank attack detection mechanism, particularly its high accuracy across both restricted and random network topologies compared to these protocols, can be attributed to the synergistic combination of its core components: DNNs and a threshold-based inconsistency-checking mechanism. This means the RankShield protocol uses a DNN model that is inherently designed to learn intricate, non-linear patterns and representations from vast amounts of data. In the context of routing attacks, this means the DNN within RankShield can identify subtle anomalies, deviations, and malicious ranks that simpler, rule-based, or less sophisticated ML models (likely employed by SBIDS and ARM) might miss.

RPL with Attack (Baseline) in Fig. 11 represents the scenario without an active detection mechanism. Its low accuracy (35.5%) is expected, as it simply reflects the impact of attacks on an unprotected network, highlighting the necessity of effective detection. The SBIDS and ARM protocols, while offering improvements (i.e., an accuracy level of 91.4% and 82% respectively) over baseline RPL with attack, likely employ less sophisticated detection approaches. The results for random topologies are less than restricted topologies suggest that their mechanisms might struggle with the inherent unpredictability and diverse traffic patterns of random topologies. They rely on simpler heuristic rules that are not as adept at discerning subtle attack patterns or adapting to varying network conditions as RankShield's dual-layer validation. The lack of a robust, adaptable learning component like a DNN, combined with potentially refined validation mechanisms, explains their lower accuracy compared to RankShield.

When considering dense network topologies in Fig. 11, the RankShield algorithm, leveraging its DNN model and threshold-based inconsistency-checking, maintains its superior performance. The increased node count and higher density, while introducing more variability than restricted topologies, would still allow the DNN to effectively learn complex normal behavior profiles due to the sheer volume of data available for training. The dual-layer validation mechanism would be crucial in filtering subtle anomalies indicative of attacks amidst the increased network traffic and interactions, enabling RankShield to achieve high accuracy, thereby outperforming RPL, SBIDS, and ARM.

5.3.6. Precision

Fig. 12 shows the remarkable 100% precision achieved by our proposed RankShield protocol, which integrates DNNs with a threshold-based inconsistency-checking mechanism, is primarily attributable to the rigorous filtering and validation provided by the threshold-based system. While the threshold system is crucial for eliminating false positives, the DNN plays a fundamental role in enabling this perfect precision. The DNN's advanced learning capabilities allow it to accurately discern intricate and subtle attack patterns within the complex RPL routing data. This means the initial set of "potential" positives provided by the DNN is already highly relevant and contains very few inherent misclassifications, making the job of the subsequent threshold filter more effective. Moreover, the DNN model that we have used in our research ensures that actual rank attacks are not missed in the first place, providing a solid foundation for the threshold mechanism to then validate. This two-stage validation process is designed to be extremely conservative, ensuring that only the most certain and unambiguous attack instances are classified as positive. This effectively filters out any benign fluctuations or ambiguous patterns that the DNN might initially flag, thereby driving the false positive rate to zero.

Even a baseline RPL without explicit attack detection might exhibit some level of "precision" if we consider its inherent behavior under certain conditions. However, a 96.2% precision for restricted network topologies suggests that while it correctly identifies most true positives, it still has a small percentage of false positives, meaning some non-attack events are mistakenly flagged as attacks. This is significantly lower than RankShield's perfect score, indicating the absence of a robust filtering mechanism.

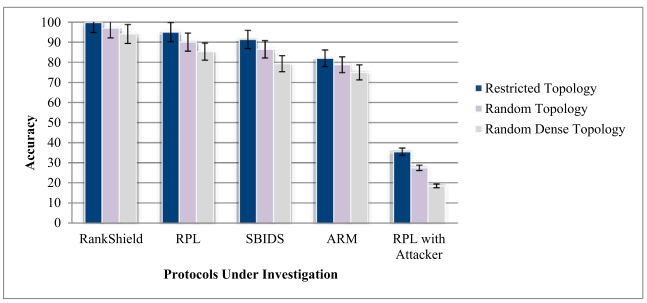


Fig. 11. Accuracy model, considering different network topologies.

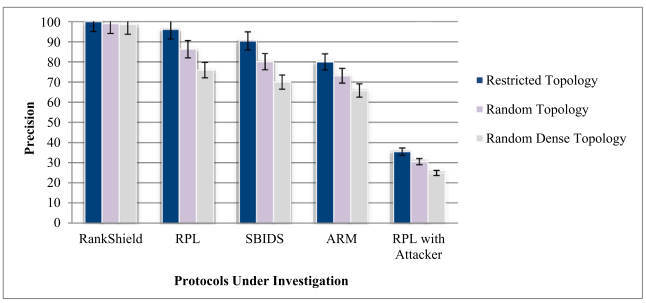


Fig. 12. Precision, considering different network topologies.

Fig. 12 show that RankShield demonstrated high precision in classifying both normal and attack instances. SBIDS and ARM shows lower precision than RankShield. The random topology is less than restricted topologies implies that SBIDS's and ARM's performance, particularly its precision, is likely more compromised in random topologies. In these less structured environments, without the sophisticated pattern recognition of a DNN and the stringent validation of a threshold system like RankShield's, SBIDS and ARM are more prone to misclassifying normal, highly variable traffic as malicious, leading to a higher rate of false positives. This further suggests that their detection mechanism is less effective at distinguishing true attacks from benign events, especially in dynamic or less predictable random topologies.

The baseline scenario (for RPL with attack), representing RPL operating under attack without any dedicated detection mechanism, naturally has the lowest precision. A 30% precision for random network topology means a substantial portion of what it might predict as an attack (if it had any inherent detection) would actually be false alarms, or it simply reflects the chaotic state where many non-attack events are indistinguishable from actual attacks.

Considering random dense network topologies in Fig. 12, RankShield's precision would maintain its exceptional 98.6%. The DNN's robust ability to identify intricate attack patterns, coupled with the stringent threshold-based validation, is highly scalable and remains effective even with a larger number of nodes and increased traffic complexity. Conversely, the precision of SBIDS, ARM, and baseline RPL would likely degrade further in such a dense environment. The increased variability and potential for ambiguous traffic in a 100-node random network topology would amplify their existing limitations, leading to a higher rate of false positives compared to the 40-node random or 19-node restricted topologies.

5.3.7. Recall

Fig. 13 shows that the proposed RankShield protocol demonstrates an exceptionally high recall rate of almost 99.5% across both restricted and random network topologies. This outstanding performance is a direct result of its innovative hybrid architecture, which leverages the strengths of DNNs and a threshold-based inconsistency-checking mechanism for detecting rank attacks. The core strength of RankShield's high recall lies in its DNN component. DNNs are highly adept at learning complex, non-linear patterns and subtle anomalies that characterize various types of rank attacks (i.e., both increased and decreased rank attacks). By being able to discern even slight deviations from normal behavior, the DNN ensures that a wide net is cast, making it highly sensitive to the presence of an attack. This capability minimizes the occurrence of false negatives (i.e., actual rank attacks being missed), directly contributing to the near-perfect recall. The adaptability of DNNs allows RankShield to maintain this high detection sensitivity even across diverse network structures, from predictable restricted topologies to more dynamic and unpredictable random network topologies.

While RPL and SBIDS offers a respectable recall of 90.1% and 86.5% respectively, it still falls short of RankShield's near-perfect performance for restricted network topologies. The note that RPL's and SBIDS's analysis emphasizes random less than restricted topologies implies that its detection methodology is optimized for, or perform better in, specific types of environments. However, without the advanced pattern recognition and adaptive filtering capabilities of RankShield's DNN and threshold system, it likely misses more subtle or rank attack instances compared to RankShield, especially across varying topological complexities. ARM demonstrates a lower recall than SBIDS as shown in Fig. 13, indicating that it misses an even larger percentage of actual attacks. Similar to SBIDS, its emphasis on random over restricted topologies in its analysis might suggest limitations in consistently identifying attacks across different network structures, and its detection logic is simply less comprehensive than RankShield's.

In Fig. 13, RPL with Attack represents the baseline scenario of an RPL network under attack without any specific mitigation or advanced detection in place. Its low recall (26.3%) when considering random network topologies underscores the inherent vulnerability of standard RPL to rank attacks and the critical need for a robust detection mechanism like RankShield to identify the vast majority of threats. The low recall rate also indicates that a substantial proportion of actual rank attacks go undetected, leaving the network vulnerable.

The introduction of random dense network topologies, consisting of 100 nodes, further underscores the robust performance of the RankShield protocol as shown in Fig. 13. Even within these larger and more complex environments, RankShield maintains its near-perfect recall rate, consistently identifying close to 99.2% of all rank attacks. This sustained high performance highlights the scalability and adaptability of its hybrid DNN and threshold-based architecture. The DNN's ability to learn intricate attack patterns remains effective regardless of network density, ensuring that subtle anomalies are detected even amidst a greater volume of network traffic and node interactions. This demonstrates RankShield's capability to provide superior protection in real-world IoT deployments, which often feature dense and extensive networks.

5.3.8. F_1-Score

Fig. 14 shows the proposed RankShield protocol demonstrates an outstanding F_1-Score of almost 99% for restricted and 97.5% for random network topologies. This near-perfect score signifies its exceptional overall performance in detecting RPL rank attacks. Moreover, the high F_1-Score is directly attributed to the protocol's hybrid approach, which judiciously combines the strengths of DNNs with a stringent threshold-based inconsistency-checking mechanism.

Fig. 14 also reveals that RankShield's consistent high performance across both restricted and random network topologies is a testament to the adaptability of its deep learning component. DNNs can learn and generalize patterns effectively, regardless of whether the network structure is predictable (restricted) or highly variable (random). This ensures that the high F_1-Score is maintained in diverse deployment environments as shown in Fig. 14, making RankShield a highly reliable solution.

When compared to other protocols, RankShield's F1-Score highlights its superior overall effectiveness as shown in Fig. 14. The RPL protocol,

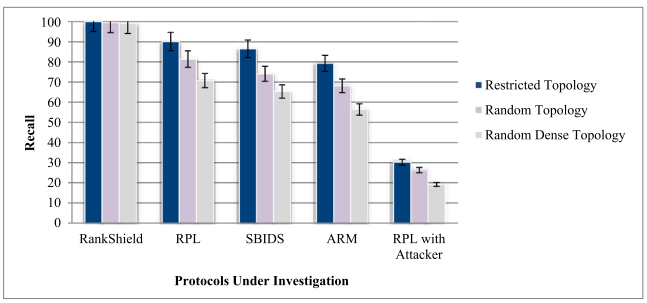


Fig. 13. Recall, considering different network topologies.

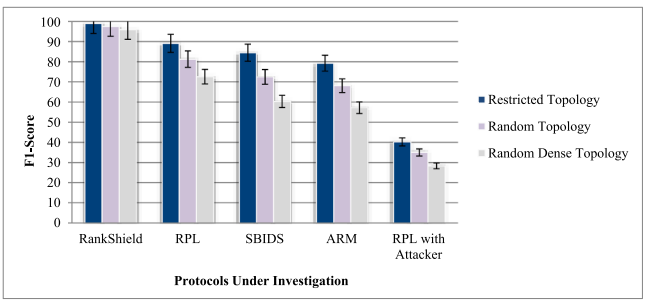


Fig. 14. F_1-Score, Considering Different Network Topologies.

with an F_1-Score of 89.1% (restricted topologies) and 81.3% (random topologies), shows a good level of overall performance. However, RankShield's 99.7% F_1-Score demonstrates a significant leap in effectiveness, indicating that our hybrid approach provides a crucial edge in balancing detection and accuracy that RPL's method likely lacks. The 16.2% difference, while seemingly small, can be critical in security applications where every missed attack or false alarm counts. Moreover, SBIDS and ARM exhibit a slightly lower F_1-Score than RankShield for restricted network topologies, at 84.5% and 79.3% respectively. The analysis focuses more on random less than restricted topologies suggests that its overall performance might be more challenged by the complexities or specific patterns found in random environments. Its lower score indicates a less optimal balance between missing attacks and generating false alarms.

RPL with attack represents the baseline performance of the RPL protocol when subjected to attacks without sophisticated detection mechanisms. Its substantially low F_1-Score of 35% underscores its poor overall effectiveness in random network topologies. This indicates that standard RPL, without dedicated security enhancements like RankShield, struggles significantly both in detecting actual attacks and avoiding misclassifications, leading to a high rate of missed threats and/or false alerts.

The proposed system achieves higher detection accuracy for both decreased and increased rank attacks by employing our custom DNNs, as clearly illustrated in the accompanying graph. This allows the system to efficiently detect RPL attacks on control packets routed through IoT devices. Consequently, our system offers superior attack detection accuracy with minimal overhead compared to other existing RPL attack-based IDSs.

When introducing dense network topologies consisting of 100 nodes, RankShield continues to demonstrate exceptional performance, achieving an F1-Score of approximately 98.2% as shown in Fig. 14. This slight variation from smaller topologies is attributed to the increased complexity and communication overhead in larger networks, which the DNN component effectively navigates due to its robust generalization capabilities. The maintenance of such a high F1-Score in dense environments further solidifies RankShield's adaptability and reliability across diverse IoT deployments, from small-scale to extensive, showcasing its superior balance of detection accuracy and false alarm reduction. This performance contrasts sharply with other protocols (SBIDS, ARM, baseline RPL, and RPL with attacker), which typically see a more significant degradation in larger, denser networks.

5.3.9. Memory consumption

Fig. 15 illustrates the average RAM usage of three rank detection protocols: SBIDS, ARM, and RankShield. This comparison considers networks with 20, 40, 60, 80, and 100 nodes, including two attacker nodes executing rank attacks (one increasing and one decreasing rank). The original RPL implementation uses 2238 bytes of RAM when the number of entries in the routing table reaches 20. SBIDS, ARM, and RankShield add an additional 78, 22, and 10 bytes, respectively, to this. This means RankShield requires a total of 2248 bytes of RAM, which is well within the 8 KB available on Zolertia or Z1 devices. The 10-byte overhead for RankShield is attributed to the need for nodes to maintain a local blacklist table. Given the Z1 sensor's total 8 KB RAM, RankShield's requirement of 2.24 KB per 20 devices makes it suitable for resource-constrained environments.

Fig. 15 also reveals a linear relationship between the number of entries in the routing table and RAM consumption on the Z1 mote. Each additional entry in the routing table, which stores neighbor IDs when in storing mode, increases RAM usage by approximately 55 bytes. Furthermore, if a rank attacker is present, nodes must also store the malicious ID in a blacklist table. RankShield demonstrates lower memory consumption compared to SBIDS and ARM because it integrates neighbor IDs into the existing routing table rather than creating a new one.

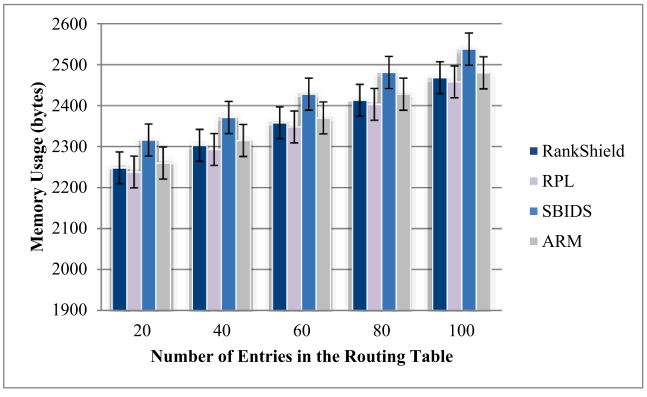


Fig. 15. RAM usage for the corresponding number of entries in the routing table.

5.3.10. Impact of traffic load

To better understand and imitate real-world IoT networks, and to more deeply analyze attack detection and mitigation protocols, it's crucial to examine how bursty or event-driven traffic affects key performance parameters. These parameters include packet delivery rate, delay, detection accuracy, and control packet overhead, particularly under random, dense network topologies. The subsequent results delve into these details.

Average PDR in terms of varying traffic load: To assess the reliability of the protocols examined, we focused on the average PDR. As depicted in Fig. 16, the average PDR varies with differing traffic loads, including bursty traffic. It's evident that all protocols perform comparably when traffic loads are below 5 packets per minute. However, RankShield consistently maintains an average PDR exceeding 98.5%, irrespective of the traffic load. This superior performance stems from its decentralized detection method, where the processing of traffic load is distributed across multiple nodes. Each node acts as a monitor, cross-referencing and evaluating incoming packets. This approach effectively mitigates congestion and prevents buffer overflows that might occur at a single monitoring node. Conversely, SBIDS, and ARM exhibit a lower PDR compared to RankShield, a difference that becomes particularly noticeable when traffic surpasses 5 packets per minute. This can be attributed to their centralized, single-step detection approaches, where a sole monitoring node is responsible for processing every incoming packet. Such a setup often leads to congestion and buffer overflows at the monitoring or sink node when network capacity is exceeded. Consequently, this results in packet loss and extended queuing delays, as packets spend more time in buffers, potentially leading to timeouts and retransmissions. In contrast, RankShield's decentralized strategy, which distributes processing and monitoring

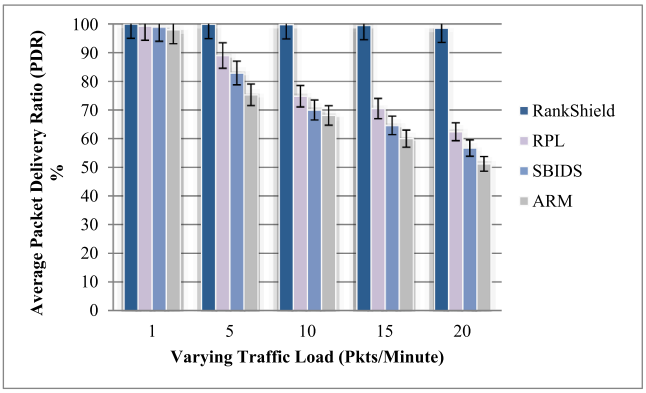


Fig. 16. Average PDR in terms of varying traffic load.

functions among all nodes in the RPL network, significantly reduces congestion and buffer overflow rates.

The superior PDR of RankShield under traffic loads of 5, 10, 15, and 20 packets/min, which is approximately 25% greater than RPL without mitigation, and almost equivalent to an attack-free environment, stems from its advanced defensive mechanisms. These include deep packet inspection, facilitated by DNNs, and a threshold-based method for identifying inconsistencies.

Average Delay in terms of varying traffic load: It's clear that RankShield consistently maintains minimal delay, even when faced with heavy traffic in a randomly dense network. However, once the traffic load surpasses 5 packets per minute, both SBIDS and ARM struggle to process the increased volume of packets at the monitoring node. This limitation stems from their reliance on a centralized approach for detecting and monitoring rank attackers within the RPL network. Such a centralized node becomes overwhelmed by the sudden influx of packets, leading to a build-up in its buffers. Consequently, incoming traffic experiences longer queuing delays, including essential protocol operation packets. Furthermore, if these queues reach full capacity, packets may be dropped, necessitating retransmissions and further exacerbating end-to-end delay.

Fig. 17 illustrates that fluctuating inter-arrival times during traffic bursts (at 5, 10, 15, and 20 Packets/Minute) lead to considerable jitter. This jitter, which refers to the variation in packet reception delays at monitoring nodes, poses challenges for centralized protocols like SBIDS and ARM. Such protocols struggle to process data consistently and predictably under these conditions. Additionally, their dependence on a single point for data aggregation can result in slower detection and response times.

Control Packets Overhead in terms of varying traffic load: Fig. 18 illustrates how control packet overhead changes with different traffic loads and bursty traffic conditions. In a dense, random network, RankShield demonstrates significantly greater efficiency in control packet overhead compared to SBIDS and ARM protocols. Its overhead is almost identical to that of RPL when no attacks are present. RankShield maintains this superior performance even when traffic loads vary (at 5, 10, 15, and 20 Packets/Minute). This is because RankShield nodes conduct local analysis during bursty traffic, reducing the necessity of sending all DIO-related data to a central location. Only summarized DAO alerts are exchanged, which results in a lower, more distributed, and localized control packet overhead. In contrast, centralized approaches like SBIDS and ARM suffer from their centralized analysis mechanisms. All incoming DIO messages from other nodes in the RPL network are funneled to a central sink node. If the link to this central sink node fails due to high traffic load or congestion, a new link or path must be re-established. This process requires additional control packet exchanges between nodes, increasing the control packet overhead compared to RankShield, which consistently minimizes it.

Accuracy in terms of varying traffic load: Fig. 19 illustrates how

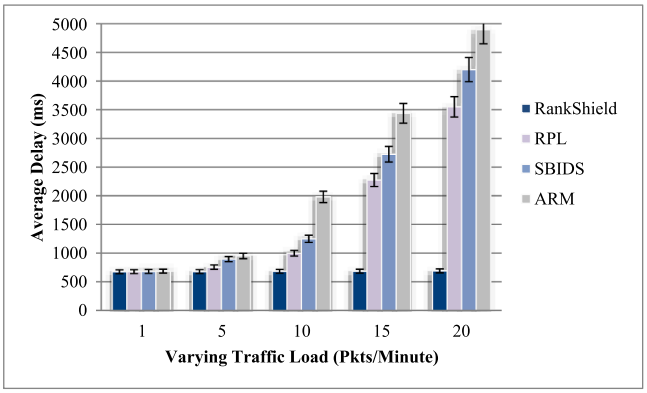


Fig. 17. Average delay in terms of varying traffic load.

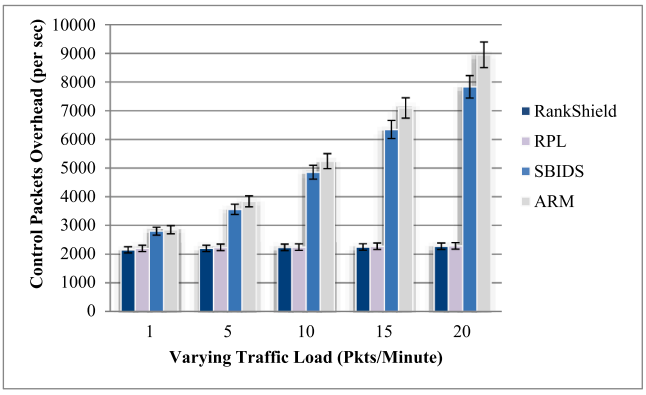


Fig. 18. Control packets overhead in terms of varying traffic load.

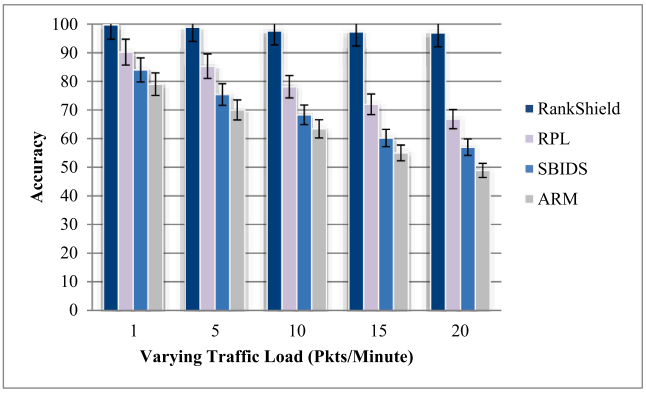


Fig. 19. Accuracy in terms of varying traffic load.

detection accuracy is affected by different traffic loads, including bursty traffic. RankShield demonstrates superior accuracy compared to SBIDS and ARM within a dense, random network topology. This high level of performance remains consistent across various traffic loads (5, 10, 15, and 20 Packets/Min) conditions. The reason for RankShield's effectiveness lies in its utilization of a DNN. The DNN's intrinsic capability to identify intricate, non-linear patterns allows it to adjust to fluctuating network conditions. This adaptability is crucial for sustaining strong performance, especially when traffic is dynamic and unpredictable, as is characteristic of bursty traffic. Consequently, even with increased burstiness, RankShield's DNN can learn accurate profiles of "normal" network behavior. This, combined with its effective threshold filtering mechanism, allows it to maintain high detection accuracy in a dense network.

In contrast, SBIDS and ARM exhibit lower accuracy with bursty traffic. This is attributed to their centralized detection approach, where a single sink node is responsible for cross-referencing, processing, and evaluating all incoming DIO packets. This centralized method leads to increased congestion and buffer overflow, ultimately resulting in reduced detection accuracy.

5.3.11. Impact of threshold

To accurately emulate dynamic IoT networks with fluctuating ranks and dense, random topologies, we must analyze the impact of varying thresholds on key performance indicators. We will investigate thresholds of 3, 6, 9, and 12, examining their effects on PDR, energy consumption, delay, control packet overhead, accuracy, and precision. The threshold dynamically adjusts with node position within the RPL network. Therefore, our experiments will utilize random threshold values ranging from 3 to 12 to thoroughly evaluate performance under these dynamic conditions.

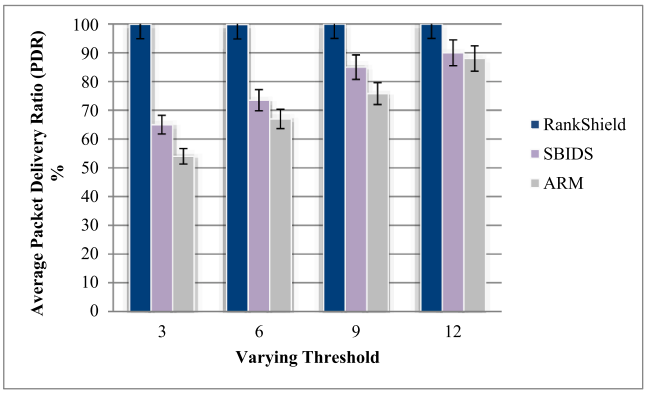


Fig. 20. Average PDR in terms of varying thresholds.

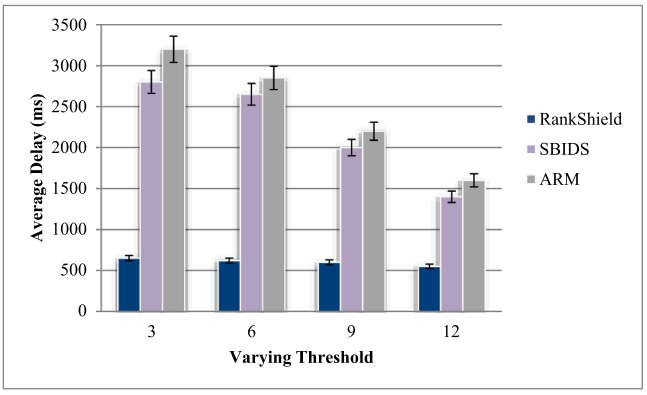


Fig. 21. Average delay in terms of varying thresholds.

PDR in terms of Varying Threshold (T): Fig. 20 illustrates the PDR across different threshold values. As shown, RankShield consistently achieves a higher PDR compared to SBIDS and ARM when operating with varying thresholds (3, 6, 9, and 12). This improvement in PDR can be attributed to RankShield's two-step detection process. Initially, DNNs filter packets, which are then cross-referenced with a dynamically adjusted threshold mechanism. This dynamic thresholding capability in RankShield ensures stable routing for nodes and minimizes rank fluctuations.

Fig. 20 additionally demonstrated that at reduced threshold values (3 and 6), both SBIDS and ARM exhibit a lower PDR. This phenomenon arises from frequent route alterations and minor rank fluctuations, which can be attributed to subtle shifts in link quality or node movement. These minor changes are enough to flag a route as "deviated," potentially initiating a route repair process or the selection of an entirely new path. Consequently, the network operates in a perpetual state of instability, continually seeking optimal routes, which results in packet loss during these transitional periods. In certain instances, if a sufficiently "stable" route cannot be established under these stringent deviation parameters, a node may become temporarily isolated and unable to transmit packets. In such circumstances, Rankshield demonstrates superior performance. This efficacy stems from Rankshield's dynamic adjustment of rank based on the neighbor's position within the RPL network, thereby diminishing the likelihood of incessant rank fluctuations and subsequent packet loss.

With increased threshold values (specifically 9 and 12), the PDR for both SBIDS and ARM protocols shows an improvement and greater stability, as illustrated in Fig. 20. This can be attributed to the routes being maintained for extended durations. Consequently, there are fewer instances of packet drops due to ongoing updates of the forwarding path, leading to a comparatively higher PDR for these protocols. Similarly, Rankshield also demonstrates a higher PDR at these threshold values (9 and 12). This is because Rankshield's dynamic rank adjustment mechanism makes it highly resilient to changes, which is particularly advantageous in environments with high mobility or noise where ranks frequently fluctuate.

Delay in terms of Varying Threshold (T): When the threshold variation is minimal, both SBIDS and ARM exhibit greater delays compared to RankShield. This is because SBIDS and ARM lack dynamic threshold adjustment based on rank fluctuations. Such fluctuations can trigger route repairs and frequent route changes, increasing control packet exchanges (e.g., DIO, DIS and DAO messages) to establish new routes, consequently increasing delay. However, with larger threshold values (e.g., 9 and 12), SBIDS and ARM experience a 50% reduction in delay. This is attributed to minor network fluctuations, which decrease the number of control packets exchanged. As a result, network establishment and packet transmission initiation occur with reduced delay.

RankShield consistently achieves reduced delays, as illustrated in Fig. 21, by dynamically adapting its threshold in response to fluctuations in rank. This adaptive approach minimizes the need for continuous searches for optimal routes and frequent exchanges of control packets, thereby mitigating network delays.

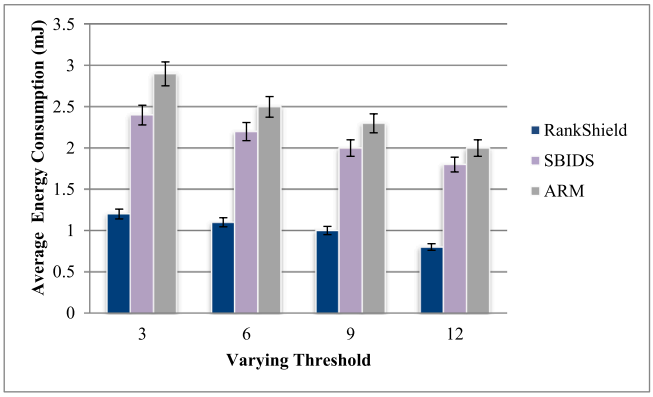


Fig. 22. Average energy consumption in terms of varying thresholds.

Energy consumption in terms of Varying Threshold (T): Fig. 22 displays the average energy consumption across different threshold values. It indicates that both SBIDS and ARM systems use more energy when the threshold variation is lower (specifically at thresholds of 3 and 6). This increased energy usage stems from nodes frequently activating within a single TSCH timeslot to exchange a greater number of control packets. This heightened activity is a consequence of significant rank fluctuations, which in turn cause frequent route changes and the need for the node to wake up and to select new communication paths. Conversely, RankShield demonstrates lower energy consumption because nodes became active less often, owing to a reduced likelihood of

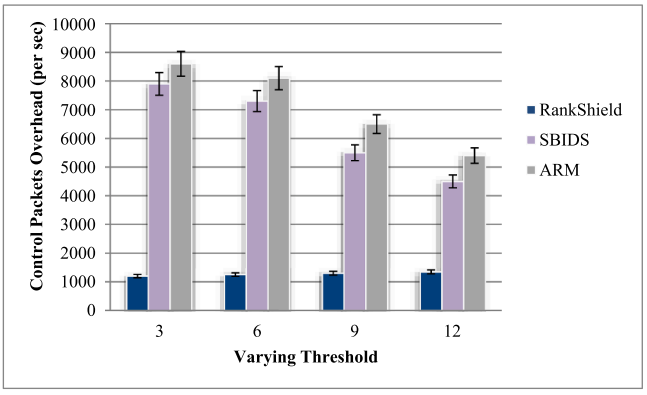


Fig. 23. Control packets overhead in terms of varying thresholds.

rank instability.

Fig. 22 illustrates that increasing the threshold to 9 and 12 enhances path reliability. This improvement can lead to a reduction in control packet exchanges, subsequently lowering energy consumption for RankShild, SBIDS, and ARM.

Control Packets Overhead in terms of Varying Threshold (T): The control packet overhead in function of varying threshold is shown in Fig. 23. Both SBIDS and ARM exhibit higher control packet overhead when the threshold variation is lower. This phenomenon occurs because a reduced threshold leads to more frequent route changes. Consequently, this necessitates a greater exchange of control packets, including DIO, DAO, and DIS messages, to establish and maintain new routes within the network.

Fig. 23 further demonstrates RankShield's consistent performance despite changes in the threshold. This stability stems from RankShield's dynamic threshold adjustment, which is based on existing rank variations, a feature absent in SBIDS and ARM, which utilize fixed thresholds. This adaptive threshold mechanism minimizes frequent route alterations and the corresponding control packet overhead in the RPL network.

Accuracy in terms of Varying Threshold (T): The acuracy as a function of the varying threshold is presented in Fig. 24. For both SBIDS and ARM, a low threshold typically leads to reduced accuracy. This occurs because the systems misinterpret normal network activities, such as routine route alterations and minor rank shifts, as "deviations." Since these protocols lack an adaptive threshold mechanism, a fixed low threshold results in a high rate of false positives, thereby diminishing their ability to accurately identify genuine threats. As the threshold increases (e.g., from 6 to 9, and then to 12), the accuracy for SBIDS and ARM generally improves. This is attributed to a higher threshold accommodating wider rank fluctuations, making it more likely for predictions to be considered "correct." Consequently, more ranks fall within the permissible deviation range as this range expands. However, a static high threshold can also impair detection accuracy. In such cases, legitimate but subtle rank variations that signal an attack might not meet the stringent criteria, leading to missed detections.

Fig. 24 illustrates RankShield's superior performance over SBIDS and ARM protocols in a random dense network topology, a trend that holds true across various threshold settings (3, 6, 9, and 12). RankShield's advantage stems from its use of a dynamically adjusted threshold. This adaptive approach enables more precise detection by accounting for normal fluctuations in rank while effectively pinpointing actual deviations. Conversely, SBIDS and ARM, which rely on static thresholds, are prone to false positives when thresholds are set too low and miss legitimate traffic when thresholds are too high. RankShield's dynamic threshold mechanism allows it to consistently achieve higher accuracy.

Precision in terms of Varying Threshold (T): The precision as a function of the varying threshold is presented in Fig. 25. When thresholds are set low (for values of 3 and 6), both SBIDS and ARM systems

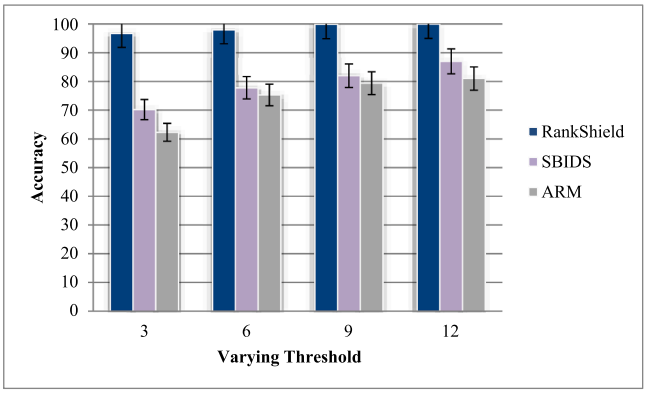


Fig. 24. Accuracy in terms of varying thresholds.

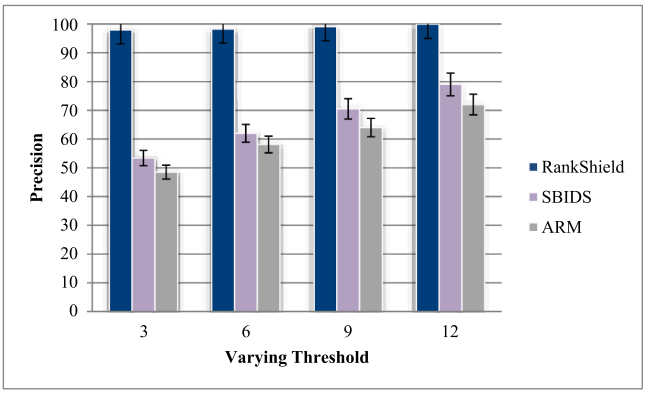


Fig. 25. Precision in terms of varying thresholds.

demonstrate reduced precision. This is attributed to their lack of an adaptive thresholding mechanism, causing them to mistakenly identify minor, routine fluctuations in rank as "deviations." Consequently, this leads to an elevated rate of false positives, where normal network operations are incorrectly flagged as malicious activity. When a large number of benign occurrences are misclassified as threats, the ratio of genuine threats to total alerts diminishes, thereby lowering precision. Conversely, as thresholds are increased (specifically from 6 to 9 and 12), the precision of SBIDS and ARM generally improves. A higher threshold enforces stricter detection criteria, ensuring that only more substantial rank deviations are identified. This, in turn, helps to decrease the incidence of false positives.

RankShield consistently achieves higher precision than SBIDS and ARM, a trend observed across various threshold values (3, 6, 9, and 12), as illustrated in Fig. 25. This enhanced performance stems from RankShield's use of a dynamically adjusted threshold. This adaptive capability enables the protocol to account for legitimate variations in network behavior, preventing the misclassification of normal fluctuations as threats and significantly reducing false positives. Concurrently, RankShield's dynamic nature allows it to accurately identify genuine deviations, ensuring the detection of actual malicious activities. By employing a dynamic threshold, RankShield circumvents the limitations of static threshold protocols. It avoids the reduced precision caused by numerous false positives at low static thresholds and prevents missed detections that can occur with high static thresholds. Consequently, RankShield maintains consistently high precision by accurately differentiating between normal network dynamics and actual security threats.

5.3.12. *Computation overhead*

The primary factor contributing to the overall computational cost is the training of DNN models, which represents the most demanding set of operations compared to SBIDS and ARM protocols. The evaluation of the performance of these compute-intensive DNN operations is conducted

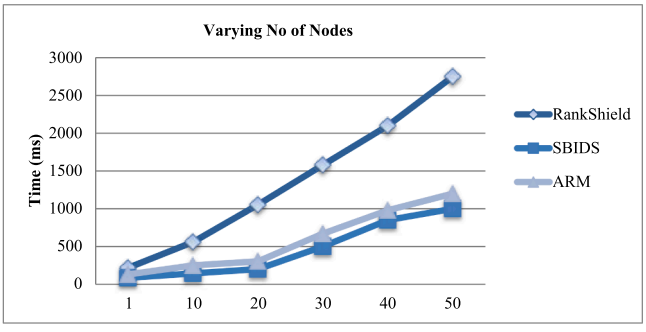


Fig. 26. Computational complexity of the RankShield's DNN detection approach for Z1 sensor with varying number of IoT devices.

using Zolertia platforms configured to simulate different roles within a network topology. These roles include the root node, the intermediate node, and the leaf node. Specifically, a Zolertia device equipped with 8 KB of RAM serves as the hardware for all three node types (root, intermediate, and leaf).

Fig. 26 presents the computational complexity of the Zolertia Z1 sensor node when executing the training process for the DNN models utilized within the RankShield system. The computational cost, measured in time, varies significantly based on the number of participating devices (nodes) in the network. Specifically, the time required increases from 218 ms when training with a single device to 2750 ms when the network size reaches fifty devices. These observed computational costs are considered reasonable for the application's requirements, aligning with related studies [47]. In contrast, SBIDS and ARM exhibit consistently low computational overhead, irrespective of the number of nodes in the network. This is primarily because these protocols do not require the training or retraining of DNN models.

Regarding system scalability, it is crucial to manage latency as the number of devices grows. To maintain an acceptable delay, it is assumed that scenarios involving a significantly greater number of nodes will necessitate either the deployment of a more powerful sensor platform, such as the Zolertia Firefly sensor, or an increase in the total number of Z1 nodes to distribute the processing workload.

5.3.13. Impact of different attack densities

To accurately emulate the influence of attack patterns in RPL networks, we must analyze the impact of varying attack densities (2, 4, 6, and 8) on key performance indicators.

PDR in terms of varying attack densities: Fig. 27 illustrates that PDR generally decreases across all tested protocols as the density of attackers rises. This performance drop occurs because rank attackers sever connections within the sub-DODAG, effectively partitioning the network and blocking legitimate data from reaching the root, as well as preventing control commands from reaching target nodes. Despite this, the figure demonstrates that RankShield maintains significantly higher stability than competing techniques. Notably, with eight active attackers, the proposed RankShield framework achieves the highest PDR, exceeding RPL, SBIDS, and ARM by approximately 75%, 22%, and 32%, respectively. This resilience is attributed to RankShield's dual-phase detection and isolation strategy, which triggers immediate alerts from both the root and intermediate nodes. In contrast, other methods often rely on slower, centralized mechanisms to detect threats and broadcast alarms. As a result, RankShield minimizes the latency between detection and mitigation, preventing data loss during the critical window where other systems remain vulnerable.

Delay in terms of varying attack densities: Fig. 28 demonstrates that latency within protocols such as RPL, SBIDS, and ARM escalates as the concentration of network attackers grows. This degradation in performance occurs because rank attackers sever connections within the DODAG, effectively obstructing data flow both upstream to the root node and downstream to child nodes. Conversely, the data indicates that

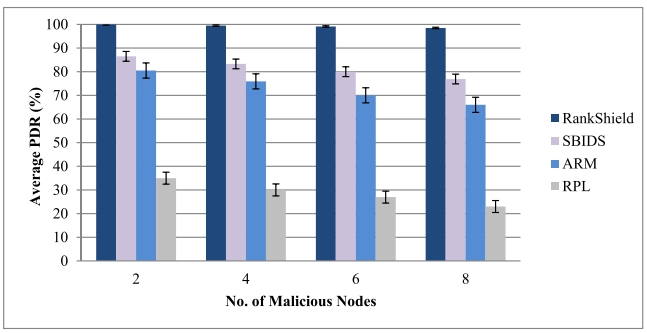


Fig. 27. PDR in terms of varying attack densities.

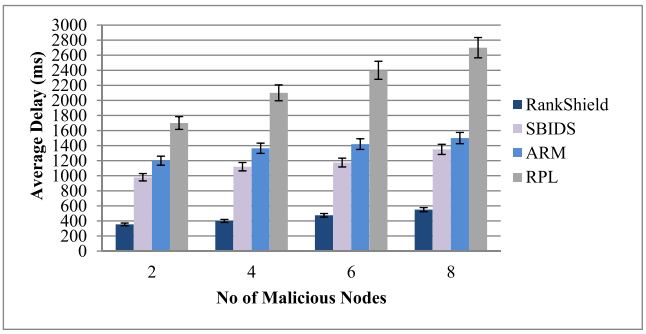


Fig. 28. Delay in terms of varying attack densities.

RankShield maintains significantly lower latency than the competing techniques. This efficiency is driven by RankShield's ability to minimize control message overhead during the route formation process. While traditional methods suffer from a surge in control packet exchange as threats increase, RankShield experiences only minor fluctuations in delay. These slight variations are simply a result of the system triggering inconsistency checks more frequently in response to adversarial presence.

Energy Consumption in terms of varying attack densities: Fig. 29 depicts the impact of increasing adversarial node density on average energy expenditure. The data reveals that the proposed RankShield protocol consistently outperforms standard RPL, which suffers from high energy costs due to the frequent transmission states required for attackers to broadcast spurious control packets and intercept traffic. Similarly, RankShield demonstrates superior efficiency compared to SBIDS and ARM. The higher energy consumption in the latter protocols stems from the absence of a robust warning mechanism; centralized nodes are forced to remain active to continuously verify incoming control packets. As the number of malicious nodes grows, SBIDS and ARM require nodes to stay awake longer for data cross-referencing, further draining power. In contrast, RankShield optimizes efficiency through a network-wide broadcast alert system. This proactive measure removes the need for redundant verification at the root nodes, allowing RankShield to sustain low energy levels even as security threats escalate.

Control Packets in terms of varying attack densities: As demonstrated in Fig. 30, control packet overhead rises across all mitigation techniques as the density of malicious nodes increases. Despite this general trend, RankShield exhibits a much more gradual rate of increase compared to protocols such as ARM, SBIDS, and RPL. This performance advantage is attributed to RankShield's ability to identify and segregate rank attackers during the early stages of DODAG topology construction, effectively limiting their ability to generate excess network traffic. Conversely, the standard RPL protocol suffers from consistently high overhead regardless of the number of adversaries, largely because

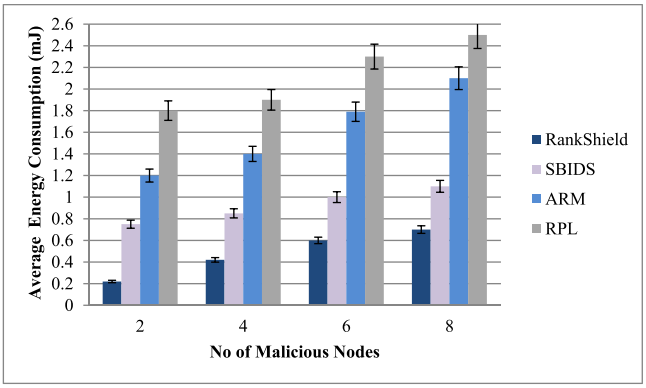


Fig. 29. Energy Consumption in terms of varying attack densities.

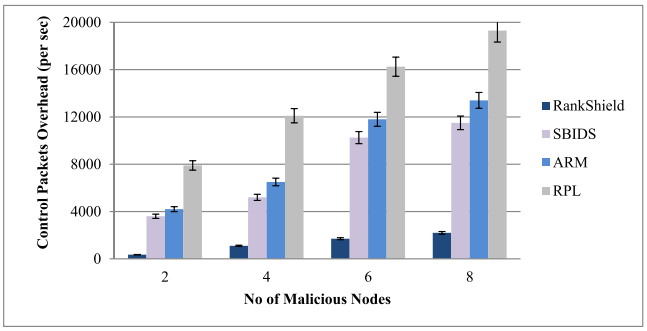


Fig. 30. Control Packets in terms of varying attack densities.

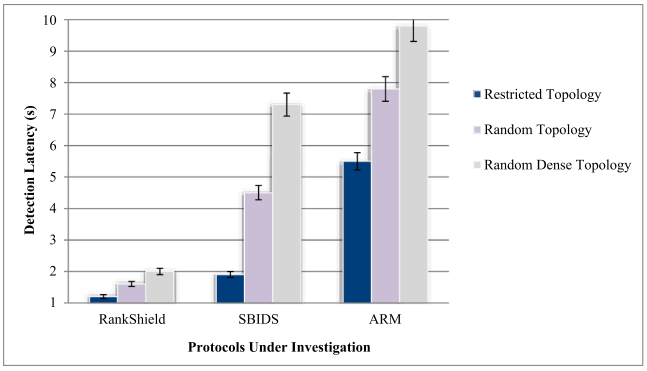


Fig. 31. Impact of detection latency, considering different network topologies.

attackers are able to continuously flood the network with deceptive DIO messages containing manipulated rank data.

5.3.14. *Detection latency*

Fig. 31 show that RankShield demonstrates a significant reduction in detection latency when compared to the SBIDS and ARM approaches. While SBIDS relies on a set of predefined rules that require multiple packet observations to confirm a rank violation, RankShield utilizes a more streamlined verification process. This allows our scheme to flag rank inconsistencies almost immediately after the malicious DIO messages are broadcast. Consequently, RankShield maintains a tighter security bound, ensuring that attackers have less time to disrupt the network topology. The optimized monitoring window in RankShield prevents the "wait-and-see" lag often found in rule-based IDS.

When measured against ARM, RankShield exhibits superior agility as depicted in Fig. 31, particularly as the network density increases. ARM often incurs a latency penalty due to its more complex recursive parent-check updates which consume additional processing cycles. RankShield, by contrast, leverages a more lightweight analytical model that identifies anomalies without requiring heavy signaling overhead between nodes. This architectural choice ensures that the detection delay remains relatively low as shown in Fig. 31, even under different network topologies. By minimizing the computational time at each hop, RankShield effectively thwarts rank-shifting attackers before they can successfully intercept or drop data packets.

6. Conclusion

The increasing reliance on IoT in critical domains has made securing resource-constrained networks a critical challenge, particularly against routing attacks that compromise reliability and performance. Among these threats, RPL rank attacks have posed a significant risk by manipulating routing metrics, thereby disrupting communication, degrading packet delivery, and increasing energy consumption.

In this work, RankShield was introduced as a hybrid detection framework that integrated the pattern recognition capability of deep neural networks with the precision of a threshold-based inconsistency-checking mechanism. This dual-layer design enabled RankShield to detect abnormal rank behavior and validate malicious activity prior to mitigation. Identified malicious nodes were isolated through blacklist propagation, ensuring their exclusion from subsequent routing decisions while preserving the availability of legitimate alternate parent nodes.

Simulation results demonstrated that RankShield outperformed baseline RPL and state-of-the-art intrusion detection approaches, including SBIDS and ARM. Specifically, RankShield consistently achieved a packet delivery ratio exceeding 98% across all evaluated network topologies, while reducing control overhead, end-to-end latency, and energy consumption. In addition, experimental evaluations confirmed the computational feasibility of the proposed framework, showing that it was capable of processing large-scale datasets within practical time and memory constraints on standard computing platforms. By combining data-driven learning with lightweight validation, RankShield provided a robust and adaptable defense mechanism for RPL-based IoT networks.

Future work will focus on extending RankShield to address additional RPL-specific attacks and evaluating its scalability in mobile and large-scale IoT deployments. Although the proposed framework was designed and experimentally evaluated for both increased and decreased rank attacks, its modular architecture enables straightforward extension to other RPL threats, including version number, sinkhole, Sybil, forwarding misbehavior, DAO inconsistency, Hatchetman, suppression, energy depletion, spam DIS, and vampire attacks. Furthermore, the impact of node mobility and frequent topology changes on detection accuracy and threshold validation will be investigated using real Zolertia sensor platforms.

Finally, while the Cooja simulator proved effective for emulating Zolertia (Z1) motes, it exhibited limitations in accurately capturing real-world operating conditions. Factors such as channel variability, clock drift, RF interference, memory constraints, environmental effects, packet loss, and physical-layer impairments are difficult to reproduce in simulation environments. To address these limitations, future evaluations will incorporate experiments on physical Zolertia (Z1) motes, enabling a more realistic assessment of RankShield's performance under real-world conditions.

CRediT authorship contribution statement

Eden Teshome Hunde: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Shereen Ismail:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Methodology, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors are unable or have chosen not to specify which data has been used.

References

[1] Z.A. Almusaylim, N.Z. Jhanjhi, A. Alhumam, Detection and mitigation of RPL rank and version number attacks in the internet of things: SRPL-RP, Sensors (Switzerland) 20 (21) (2020) 1–25, https://doi.org/10.3390/s20215997.
[2] B. Ali, G. Chen, Proactive and privacy-preserving defense for DNS over HTTPS via federated AI attestation (PAFA-DoH), Neural Netw. 119 (2026) (2025).

[3] B. Ali, G. Chen, Next-generation AI for advanced threat detection and security enhancement in DNS over HTTPS, J. Netw. Comput. Appl. 244 (2025) (2025).

[4] B. Ali, G. Chen, Neuromorphic quantum adversarial learning (NQAL): a bio-inspired paradigm for DNS over HTTPS threat detection, EURASIP J. Inf. Secur. (2025).

[5] B. Ali, On the fog's frontline: a federated machine learning approach for industrial network threat detection and intrusion prevention, J. Cybersecur. (2025).

[6] T. Tsvetkov and A. Klein, "RPL: iPv6 routing protocol for low power and lossy networks", doi: 10.2313/NET-2011-07-1_09.

[7] T.T.K. M, S.I.K. Iuchi, Secure parent node selection scheme in route construction to exclude attacking nodes from RPL network, IEICE Commun. Express 4 (2015) 340–345.

[8] A. Althubaity, H. Ji, T. Gong, M. Nixon, R. Ammar, S. Han, ARM: a hybrid specification-based intrusion detection system for rank attacks in 6TiSCH networks, in: 22nd IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), IEEE, 2017, https://doi.org/10.1109/ETFA.2017.8247593. Accessed: Jun. 07, 2025. [Online]. Available.

[9] V. Neerugatti, A.R.M. Reddy, Machine learning based technique for detection of rank attack in RPL based internet of things networks, Int. J. Innov. Technol. Explor. Eng. 8 (9) (2019) 244–248, https://doi.org/10.35940/ijitee.I3044.0789S319. Special Issue 3.

[10] S. Shin, K. Kim, T. Kwon, Detection of malicious packet dropping attacks in RPL-based internet of things, Int. J. Ad Hoc Ubiquitous Comput. 31 (2) (2019) 133–141, https://doi.org/10.1504/IJAHUC.2019.100085.

[11] Shahid Raza, Linus Wallgren, Thiemo Voigt, SVELTE: real-time intrusion detection in the Internet of Things, Ad Hoc Netw. 11 (8) (2013) 2661–2674.

[12] S. Karmakar, J. Sengupta, SD LEADER bit, low overhead rank attack detection for securing RPL based IoT, in: International Conference on COMmunication Systems & NETworkS (COMSNETS), IEEE, 2021, pp. 429–437.

[13] A. Raoof, A. Matrawy, C.H. Lung, Routing attacks and mitigation methods for RPL-based internet of things, IEEE Commun. Surv. Tutor. 21 (2) (2019) 1582–1606, https://doi.org/10.1109/COMST.2018.2885894.

[14] L. Wallgren, S. Raza, T. Voigt1, Routing attacks and countermeasures in the RPL-based internet of things, Int. J. Distrib. Sens. Netw. (2013).

[15] C. Pu, Mitigating DAO Inconsistency Attack in RPL-based Low Power and Lossy Networks, IEEE, 2018.

[16] C. Pu, X. Zhou, Suppression attack against Multicast protocol in low power and lossy networks: analysis and defenses, Sensors 18 (3236) (2018).

[17] C. Pu, Sybil attack in RPL-based internet of things:analysis and defenses, IEEE Internet Things J. 7 (6) (Jun. 2020) 4937–4949.

[18] V.R. Rajasekar, S. Rajkumar, A study on impact of DIS flooding attack on RPL-based 6LowPAN network, Microprocess. Microsyst. 94 (2022) (2022).

[19] M. Rouissat, M. Belkheir, I.S. Alsukayt, A. Mokaddem, A lightweight mitigation approach against a new inundation attack in RPL-based IoT networks, Appl. Sci. 13 (10336) (2023).

[20] N. Khan, M.S. Alshehri, M.A. Khan, S. Almakdi, N. Moradpoor, A. Alazeb, S. Ullah, N. Naz, J. Ahmad, A hybrid deep learning-based intrusion detection system for IoT networks, Math. Biosci. Eng. 20 (8) (2023) 13491–13520, https://doi.org/10.3934/mbe.2023602. Jun.

[21] Y.Al. Sawafi, A. Touzene, R. Hedjam, Hybrid Deep learning-based intrusion detection system for RPL IoT Networks, J. Sens. Actuator Netw. 12 (2) (2023) 1–21, https://doi.org/10.3390/jsan12020021.

[22] U. Shahid, M.Z. Hussain, M.Z. Hasan, A. Haider, J. Ali, J. Altaf, Hybrid intrusion detection system for RPL IoT Networks using machine Learning and Deep Learning, IEEE Access 12 (2024), https://doi.org/10.1109/ACCESS.2024.3442529.

[23] S.Bakkali Wakili, A.El.H. Alaoui, Machine learning for QoS and security enhancement of RPL in IoT-enabled wireless sensors, Sens. Int. 5 (2024), https://doi.org/10.1016/j.sintl.2024.100289.

[24] R. Khan, N. Tariq, M. Ashraf, F.A. Khan, S. Shafi, A. Ali, FL-DSFA: securing RPL-based IoT networks against selective forwarding attacks using federated learning, Sensor 24 (17) (2024), https://doi.org/10.3390/s24175834.

[25] S. Yılmaz, S. Sen, E. Aydogan, Exploring and enhancing placement of IDS in RPL: a federated learning-based approach, IEEE Internet Things J. 12 (13) (2025) 22945–22961, https://doi.org/10.1109/JIOT.2025.3553194.

[26] P.S. Nandhini, S. Kuppuswami, S. Malliga, Energy efficient thwarting rank attack from RPL based IoT networks: a review, Mater.: Proc. 81 (2) (2023) 694–699, https://doi.org/10.1016/j.matpr.2021.04.167.

[27] H. Albinali, F. Azzedin, Toward RPL attacks and Mitigation Taxonomy: systematic literature Review approach, IEEE Trans. Netw. Serv. Manag. 21 (5) (2024) 5215–5238, https://doi.org/10.1109/TNSM.2024.3386468.

[28] I.S. Alsukayti, M. Alreshoodi, RPL-based IoT networks under simple and complex routing security attacks: an experimental study, Appl. Sci. 13 (8) (2023), https://doi.org/10.3390/app13084878.

[29] R. Stephen, L. Arockiam, E2V: techniques for detecting and mitigating rank inconsistency attack (RInA) in RPL based internet of things, J. Phys. Conf. Ser. 1142 (1) (2018), https://doi.org/10.1088/1742-6596/1142/1/012009.

[30] P.S. Nandhini, S. Kuppuswami, S. Malliga, R. DeviPriya, A lightweight energy-efficient algorithm for mitigation and isolation of internal rank attackers in RPL based internet of things, Comput. Netw. 218 (2022), https://doi.org/10.1016/j.comnet.2022.109391.

[31] G. Glissa, A. Rachedi, A. Meddeb, A secure routing protocol based on RPL for Internet of things, in: 2016 IEEE Global Communications Conference (GLOBECOM), IEEE, 2016, https://doi.org/10.1109/GLOCOM.2016.7841543. Accessed: Jun. 07, 2025. [Online]Available.

[32] D. Airehrour, J.A. Gutierrez, S.K. Ray, SecTrust-RPL: a secure trust-aware RPL routing protocol for Internet of Things, Future Gener. Comput. Syst. 93 (2019) 860–876, https://doi.org/10.1016/j.future.2018.03.021.

[33] A. Le, J. Loo, Y. Luo, A. Lasebae, Specification-based IDS for securing RPL from topology attacks. IFIP Wireless Days (WD), IEEE, 2011, p. 423, https://doi.org/10.1109/WD.2011.6098218. Accessed: Jun. 07, 2025[Online]Available.

[34] V. Kiran, S. Rani, P. Singh, Towards a light weight routing security in IoT using non-cooperative game models and Dempster–Shaffer theory, Wirel Pers. Commun. 110 (4) (2020) 1729–1749, https://doi.org/10.1007/s11277-019-06809-w.

[35] A.D. Seth, S. Biswas, A.K. Dhar, Detection and verification of decreased rank attack using round-trip times in RPL-based 6LoWPAN networks, in: International Symposium on Advanced Networks and Telecommunication Systems, ANTS, IEEE Computer Society, 2020, https://doi.org/10.1109/ANTS50601.2020.9342754.

[36] A.M. Said, A. Yahyaoui, F. Yaakoubi, T. Abdellatif, Machine learning based rank attack detection for smart hospital infrastructure, in: 18th International Conference, ICOST, Hammamet, Tunisia, 2020, pp. 28–40.

[37] M.M. Iqbal, A. Ahmed, U. Khadam, Sinkhole attack in multi-sink paradigm: detection and performance evaluation in RPL based IoT, in: International Conference on Computing and Information Technology (ICCIT-1441), IEEE, Tabuk, Saudi Arabia, 2020.

[38] W. Choukri, H. Lamaazi, N. Benamar, RPL rank attack detection using Deep Learning, in: International Conference on Innovation and Intelligence for Informatics, Computing and Technologies (3ICT), 2020.

[39] F. Zahra, N.Z. Jhanjhi, S.N. Brohi, N.A. Khan, M. Masud, M.A. Alzain, Rank and wormhole attack detection model for RPL-based internet of things using machine learning, Sensors 22 (18) (2022), https://doi.org/10.3390/s22186765.

[40] T. Raghavendra, M. Anand, M. Selvi, K. Thangaramya, SVN.Santhosh Kumar, A. Kannan, An intelligent RPL attack detection using machine learning-based intrusion detection system for internet of things, in: 4th International Conference on Innovative Data Communication Technology and Application, 2022.

[41] B. Ghaleb, A. Al-Dubai, A. Hussain, J. Ahmad, I. Romdhani, Z. Jaroucheh, Resolving the decreased rank attack in RPL's IoT networks, Netw. Internet Archit. (2023) [Online]. Available, http://www.ieee.org/publications_standards/publications/rights/index.html.

[42] M.A. Alqarni, S.H. Chauhdary, A security scheme for statistical anomaly detection and the mitigation of rank attacks in RPL networks (IoT Environment), Eng. Technol. Appl. Sci. Res. 13 (6) (2023) 12409–12414, https://doi.org/10.48084/etasr.6433.

[43] T. Mehmood, "COOJA network Simulator: exploring the infinite possible ways to compute the performance metrics of IOT based smart devices to understand the working of IOT based compression & routing protocols.".

[44] F.Y. Yavuz, U. Devrim, G. Ensar, Deep learning for detection of routing attacks in the internet of things, Int. J. Comput. Intell. Syst. 12 (1) (2018) 39–58.

[45] P.O. Kasinathan, C. Pastrone, M.A. Spirito, D. Viretto, Denial-of-service architectures for wireless sensor networks: 6LoWPAN-IPv6-RPL case study, in: 2013 IEEE 9th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), Lyon, France, 2013, https://doi.org/10.1109/WiMOB.2013.6673419.

[46] J. Eriksson, F. Osterlind;, T. Voigt;, N. Finne;, S. Raza, N. Tsiftes, Demo abstract: accurate power profiling of sensornets with the COOJA/MSPsim simulator, in: IEEE 6th International Conference on Mobile Adhoc and Sensor Systems, IEEE, Macau, China, 2009.

[47] Z. Yuan, K.L.Eddie Law, Distributed TinyML on resource-constrained IoT sensor networks, in: 2024 IEEE 10th World Forum on Internet of Things (WF-IoT), Ottawa, ON, Canada, 2024, pp. 457–462, https://doi.org/10.1109/WF-IoT62078.2024.10811277.